

```
tuổi trung bình của nhà ở: 0.102175
hộ gia đình 0,071426
tổng số phòng ngủ: 0,054635
dân số -0,020153
số người trên mỗi hộ gia đình -0,038224
kinh độ -0,050859
vĩ độ -0.139584
tỷ lệ phòng ngủ -0,256397
Tên: median_house_value, kiểu dữ liệu: float64
```

Không tệ chút nào! Thuộc tính bedrooms_ratio mới tốt hơn nhiều.
có mối tương quan với giá trị nhà trung bình hơn là tổng số phòng hoặc
phòng ngủ. Rõ ràng những ngôi nhà có tỷ lệ phòng ngủ/phòng thấp hơn thường có xu hướng...
đắt hơn. Số lượng phòng mỗi hộ gia đình cũng nhiều hơn.
Thông tin này hữu ích hơn tổng số phòng trong một quận—rõ ràng là vậy.
Nhà càng lớn thì giá càng cao.

Vòng thăm dò này không cần phải hoàn toàn kỹ lưỡng; điểm mấu chốt là...
Điều quan trọng là phải bắt đầu một cách thuận lợi và nhanh chóng thu thập được những thông tin hữu ích giúp bạn.
Tạo ra một nguyên mẫu đầu tiên tương đối tốt. Nhưng đây là một quá trình lặp đi lặp lại: một khi
Khi bạn có một nguyên mẫu hoạt động, bạn có thể phân tích kết quả đầu ra của nó để thu được nhiều thông tin hơn.
Những hiểu biết sâu sắc và hãy quay lại bước khám phá này sau.

Chuẩn bị dữ liệu cho học máy

Thuật toán

Đã đến lúc chuẩn bị dữ liệu cho các thuật toán Học máy của bạn. Thay vào đó
Thay vì thực hiện việc này thủ công, bạn nên viết các hàm cho mục đích này.
một số lý do chính đáng:

- Điều này sẽ cho phép bạn dễ dàng tái tạo các phép biến đổi này trên bất kỳ thiết bị nào.
tập dữ liệu (ví dụ: lần tiếp theo bạn nhận được một tập dữ liệu mới).
- Bạn sẽ dần dần xây dựng một thư viện các hàm chuyển đổi mà bạn
Có thể tái sử dụng trong các dự án tương lai.
- Bạn có thể sử dụng các chức năng này trong hệ thống đang hoạt động của mình để chuyển đổi cái mới.
dữ liệu trước khi đưa vào thuật toán của bạn.

- Điều này sẽ giúp bạn dễ dàng thử nghiệm nhiều phép biến đổi khác nhau và xem sự kết hợp nào mang lại kết quả tốt nhất.

Nhưng trước tiên, hãy quay lại tập dữ liệu huấn luyện sạch (bằng cách sao chép `strat_train_set` một lần nữa). Chúng ta cũng hãy tách các biến dự đoán và nhãn, vì chúng ta không nhất thiết muốn áp dụng cùng một phép biến đổi cho các biến dự đoán và các giá trị mục tiêu (lưu ý rằng `drop()` tạo một bản sao của dữ liệu và không ảnh hưởng đến `strat_train_set`):

```
housing = strat_train_set.drop("median_house_value", axis=1)
housing_labels = strat_train_set["median_house_value"].copy()
```

Làm sạch dữ liệu

Hầu hết các thuật toán Học máy không thể hoạt động với các đặc trưng bị thiếu, vì vậy chúng ta hãy tạo một vài hàm để xử lý chúng. Chúng ta đã thấy trước đó rằng

Thuộc tính `total_bedrooms` có một số giá trị bị thiếu, vì vậy chúng ta hãy khắc phục điều này.

Bạn có ba lựa chọn:

1. Loại bỏ các quận tương ứng.
2. Loại bỏ toàn bộ thuộc tính.
3. Gán các giá trị bị thiếu bằng một giá trị nào đó (số không, giá trị trung bình, giá trị trung vị, v.v.). Quá trình này được gọi là điền giá trị thiếu.

Bạn có thể dễ dàng thực hiện điều này bằng cách sử dụng các phương thức `dropna()`, `drop()` và `fillna()` của `DataFrame`:

```
housing.dropna(subset=["total_bedrooms"], inplace=True) # tùy chọn 1
```

```
housing.drop("total_bedrooms", axis=1) # tùy chọn 2
```

```
trung_vị = housing["total_bedrooms"].median() # tùy chọn 3
housing["total_bedrooms"].fillna(median, inplace=True)
```

Chúng ta hãy chọn phương án 3 vì nó ít gây ảnh hưởng nhất, nhưng thay vì đoạn mã ở trên, chúng ta sẽ sử dụng một lớp tiện dụng của Scikit-Learn: `SimpleImputer`.

Lợi ích là nó sẽ lưu trữ giá trị trung vị của mỗi thuộc tính: điều này sẽ làm cho nó trở nên...

Có thể điền các giá trị bị thiếu không chỉ trên tập huấn luyện mà còn trên các tập dữ liệu khác.

Tập dữ liệu xác thực, tập dữ liệu kiểm thử và bất kỳ dữ liệu mới nào được đưa vào mô hình. Đây là

Cách sử dụng. Đầu tiên, bạn cần tạo một thể hiện của `SimpleImputer`.

Chỉ định rằng bạn muốn thay thế các giá trị bị thiếu của mỗi thuộc tính bằng...

giá trị trung vị của thuộc tính đó:

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
```

Vì giá trị trung vị chỉ có thể được tính trên các thuộc tính số, bạn cần

để tạo một bản sao dữ liệu chỉ với các thuộc tính số (điều này sẽ

(loại trừ thuộc tính văn bản `ocean_proximity`):

```
housing_num = housing.select_dtypes(include=[np.number])
```

Giờ đây, bạn có thể điều chỉnh mô hình `imputer` cho phù hợp với dữ liệu huấn luyện bằng cách sử dụng...

Phương thức `fit()`:

```
imputer.fit(housing_num)
```

Máy tính chỉ đơn giản là tính toán giá trị trung vị của từng thuộc tính và

đã lưu trữ kết quả trong biến thể hiện `statistics_` của nó. Chỉ có

Thuộc tính `total_bedrooms` có giá trị bị thiếu, nhưng chúng ta không thể chắc chắn.

Điều đó có nghĩa là sẽ không có bất kỳ giá trị nào bị thiếu trong dữ liệu mới sau khi hệ thống hoạt động.

vì dữ liệu đang được xử lý trực tiếp nên việc áp dụng phương pháp nhập liệu cho tất cả các thuộc tính số sẽ an toàn hơn:

```
>>> imputer.statistics_
mảng([-118.51 , 34.26  3.5385])      , 29.   , 2125.   , 434.   , 1167.   , 408.   ,
>>> housing_num.median().values
mảng([-118.51 ,      34.26 , 29.   , 2125.   , 434.   , 1167.   , 408.   ,
      3.5385])
```

Giờ đây, bạn có thể sử dụng máy tính "đã được huấn luyện" này để biến đổi tập dữ liệu huấn luyện bằng cách...

Thay thế các giá trị bị thiếu bằng các giá trị trung vị đã học:

```
X = imputer.transform(housing_num)
```

Các giá trị bị thiếu cũng có thể được thay thế bằng giá trị trung bình (`strategy="mean"`), hoặc bằng giá trị xuất hiện thường xuyên nhất (`strategy="most_frequent"`), hoặc bằng một giá trị hằng số (`strategy="constant", fill_value=.`). Hai chiến lược cuối cùng hỗ trợ dữ liệu không phải dạng số.

MẸO

Ngoài ra, còn có các công cụ điền dữ liệu mạnh mẽ hơn có sẵn trong gói `sklearn.impute` (chỉ dành cho các đặc trưng số):

- `KNNImputer` thay thế mỗi giá trị bị thiếu bằng giá trị trung bình của k láng giềng gần nhất cho đặc trưng đó. Khoảng cách được dựa trên tất cả các đặc trưng có sẵn.
- `IterativeImputer` huấn luyện một mô hình hồi quy cho mỗi đặc trưng để dự đoán các giá trị bị thiếu dựa trên tất cả các đặc trưng khác có sẵn. Sau đó, nó huấn luyện lại mô hình trên dữ liệu đã được cập nhật và lặp lại quá trình này nhiều lần, cải thiện các mô hình và các giá trị thay thế ở mỗi lần lặp.

Thiết kế SCIKIT-LEARN

API của Scikit-Learn được thiết kế rất tốt. Đây là những nguyên tắc thiết kế **chính** :

9

Tính nhất quán

Tất cả các đối tượng đều có giao diện nhất quán và đơn giản:

Người ước tính

Bất kỳ đối tượng nào có thể ước tính một số tham số dựa trên tập dữ liệu đều được gọi là bộ ước lượng (ví dụ: SimpleImputer là một bộ ước lượng). Quá trình ước lượng được thực hiện bởi phương thức `fit()`, và nó nhận một tập dữ liệu làm tham số, hoặc hai tập dữ liệu đối với các thuật toán học có giám sát—tập dữ liệu thứ hai chứa các nhãn. Bất kỳ tham số nào khác cần thiết để hướng dẫn quá trình ước lượng đều được coi là siêu tham số (chẳng hạn như chiến lược của SimpleImputer), và nó phải được thiết lập như một biến thể hiện (thường thông qua tham số hàm tạo).

Máy biến hình

Một số thuật toán ước lượng (như SimpleImputer) cũng có thể biến đổi tập dữ liệu; chúng được gọi là các bộ biến đổi (transformers). Một lần nữa, API rất đơn giản: phép biến đổi được thực hiện bằng phương thức `transform()` với tập dữ liệu cần biến đổi làm tham số. Phương thức này trả về tập dữ liệu đã được biến đổi. Phép biến đổi này thường dựa trên các tham số đã học, giống như trường hợp của SimpleImputer. Tất cả các bộ biến đổi cũng có một phương thức tiện lợi gọi là `fit_transform()` tương đương với việc gọi `fit()` rồi `transform()` (nhưng đôi khi `fit_transform()` được tối ưu hóa và chạy nhanh hơn nhiều).

Các yếu tố dự báo

Cuối cùng, một số bộ ước lượng, khi được cung cấp một tập dữ liệu, có khả năng đưa ra dự đoán; chúng được gọi là các bộ dự báo. Ví dụ, mô hình Hồi quy tuyến tính trong chương trước là một bộ dự báo: khi được cung cấp GDP bình quân đầu người của một quốc gia, nó dự đoán mức độ hài lòng về cuộc sống. Một bộ dự báo có phương thức `predict()` nhận một tập dữ liệu gồm các trường hợp mới và trả về một tập dữ liệu gồm các dự đoán tương ứng. Nó cũng có một phương thức `score()` đo lường chất lượng của các dự đoán, khi được cung cấp một tập dữ liệu kiểm tra (và các nhãn tương ứng, trong trường hợp các thuật toán học có ¹⁰ giám sát).

Điều tra

Tất cả các siêu tham số của bộ ước lượng đều có thể truy cập trực tiếp thông qua các biến thể hiện công khai (ví dụ: `imputer.strategy`), và tất cả các tham số đã học của bộ ước lượng đều có thể truy cập thông qua các biến thể hiện công khai có hậu tố gạch dưới (ví dụ: `imputer.statistics_`).

Không gia tăng số lượng lớp học

Các tập dữ liệu được biểu diễn dưới dạng mảng NumPy hoặc ma trận thưa SciPy, thay vì các lớp tự tạo. Các siêu tham số chỉ là các chuỗi hoặc số thông thường trong Python.

Bố cục

Các khối xây dựng hiện có được tái sử dụng nhiều nhất có thể. Ví dụ, như chúng ta sẽ thấy, việc tạo ra một bộ ước lượng Pipeline từ một chuỗi các bộ chuyển đổi tùy ý, tiếp theo là một bộ ước lượng cuối cùng, là rất dễ dàng.

Các giá trị mặc định hợp lý

Scikit-Learn cung cấp các giá trị mặc định hợp lý cho hầu hết các tham số, giúp dễ dàng nhanh chóng tạo ra một hệ thống hoạt động cơ bản.

Các bộ chuyển đổi Scikit-Learn xuất ra mảng NumPy (hoặc đôi khi là ma trận thừa SciPy) ngay cả khi chúng được cung cấp DataFrame của Pandas làm đầu vào. Vì vậy, đầu ra của ¹¹ `imputer.transform(housing_num)` là một mảng NumPy: X không có tên cột cũng như chỉ mục. May mắn thay, việc gói X vào một DataFrame và khôi phục tên cột và chỉ mục từ `housing_num` không quá khó:

```
housing_tr = pd.DataFrame(X, columns=housing_num.columns,
                          index=housing_num.index)
```

Xử lý các thuộc tính văn bản và phân loại

Cho đến nay chúng ta mới chỉ xem xét các thuộc tính số, nhưng bây giờ hãy cùng xem xét các thuộc tính văn bản. Trong tập dữ liệu này, chỉ có một thuộc tính duy nhất: thuộc tính `ocean_proximity`. Hãy cùng xem giá trị của nó trong một vài trường hợp đầu tiên:

```
>>> housing_cat = housing[["ocean_proximity"]] >>> housing_cat.head(8)
ocean_proximity NEAR BAY <1H OCEAN
INLAND
13096
14973
3785
14689      NỘI ĐỊA
20507      GẦN ĐẠI DƯƠNG
1286      NỘI ĐỊA
18078      <1H ĐẠI DƯƠNG
4396      GẦN VỊNH
```

Đây không phải là văn bản ngẫu nhiên: có một số lượng giá trị khả dĩ nhất định, mỗi giá trị đại diện cho một danh mục. Vì vậy, thuộc tính này là một thuộc tính phân loại. Hầu hết các thuật toán Học máy đều thích làm việc với số, vì vậy hãy chuyển đổi các danh mục này từ văn bản sang số. Để làm điều này, chúng ta có thể sử dụng lớp `OrdinalEncoder` của Scikit-Learn:

```
from sklearn.preprocessing import OrdinalEncoder

ordinal_encoder = OrdinalEncoder() housing_cat_encoded
= ordinal_encoder.fit_transform(housing_cat)
```

Đây là một vài giá trị được mã hóa đầu tiên trong bảng `housing_cat_encoded`:

```
>>> housing_cat_encoded[:8]
array([[3.],
       [0.],
       [1.],
       [1.],
       [4.],
       [1.],
       [0.],
       [3.]])
```

Bạn có thể lấy danh sách các danh mục bằng cách sử dụng biến thể hiện `categories_`. Đó là một danh sách chứa một mảng 1 chiều các danh mục cho mỗi thuộc tính phân loại (trong trường hợp này, một danh sách chứa một mảng duy nhất vì chỉ có một thuộc tính phân loại):

```
>>> ordinal_encoder.categories_
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
      dtype=object)]
```

Một vấn đề với cách biểu diễn này là các thuật toán học máy sẽ cho rằng hai giá trị gần nhau thì tương tự nhau hơn hai giá trị xa nhau. Điều này có thể ổn trong một số trường hợp (ví dụ: đối với các danh mục được sắp xếp như “kém”, “trung bình”, “tốt” và “xuất sắc”), nhưng rõ ràng là không đúng với cột `ocean_proximity` (ví dụ: danh mục 0 và 4 rõ ràng tương tự nhau hơn danh mục 0 và 1). Để khắc phục vấn đề này, một giải pháp phổ biến là tạo một thuộc tính nhị phân cho mỗi danh mục: một thuộc tính bằng 1 khi danh mục là “<1H OCEAN” (và 0 nếu ngược lại), một thuộc tính khác bằng 1 khi danh mục là “INLAND” (và 0 nếu ngược lại), v.v. Điều này được gọi là mã hóa one-hot, bởi vì chỉ có một thuộc tính bằng 1 (nóng), trong khi các thuộc tính khác bằng 0 (lạnh). Các thuộc tính mới đôi khi được gọi là thuộc tính giả. Scikit-Learn cung cấp lớp `OneHotEncoder` để chuyển đổi các giá trị phân loại thành vectơ one-hot:


```
from sklearn.preprocessing import OneHotEncoder

cat_encoder = OneHotEncoder()
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
```

Theo mặc định, đầu ra của OneHotEncoder là một ma trận thưa SciPy, thay vì một mảng NumPy:

```
>>> housing_cat_1hot <ma
trận thưa 16512x5 kiểu '<class 'numpy.float64'>' với 16512 phần tử được lưu
trữ ở định dạng Hàng thưa nén>
```

Ma trận thưa là một cách biểu diễn rất hiệu quả cho các ma trận chủ yếu chứa các phần tử bằng 0. Thật vậy, bên trong nó chỉ lưu trữ các giá trị khác 0 và vị trí của chúng. Khi một thuộc tính phân loại có hàng trăm hoặc hàng nghìn danh mục, thì việc mã hóa one-hot sẽ tạo ra một ma trận rất lớn chứa đầy các phần tử bằng 0 ngoại trừ một phần tử 1 duy nhất trên mỗi hàng. Trong trường hợp này, ma trận thưa chính xác là những gì chúng ta cần: nó sẽ tiết kiệm rất nhiều bộ nhớ và tăng tốc độ tính toán.

Bạn có thể sử dụng ma trận thưa gần giống như một mảng 2D thông thường, ¹² nhưng nếu bạn muốn chuyển đổi nó thành một mảng NumPy (dày), chỉ cần gọi phương thức `toarray()`:

```
>>> housing_cat_1hot.toarray()
array([[0., 0., 0., 1., 0.],
       [1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       ...,
       [0., 0., 0., 0., 1.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1.]])
```

Ngoài ra, bạn cũng có thể đặt `sparse=False` khi tạo OneHotEncoder, trong trường hợp đó phương thức `transform()` sẽ trả về trực tiếp một mảng NumPy thông thường (dense).

Tương tự như với `OrdinalEncoder`, bạn có thể lấy danh sách các danh mục bằng cách sử dụng biến thể `categories_` của bộ mã hóa:

```
>>> cat_encoder.categories_
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR
OCEAN'],
      dtype=object)]
```

Pandas có một hàm gọi là `get_dummies()` cũng chuyển đổi mỗi đặc trưng phân loại thành dạng biểu diễn one-hot, với một đặc trưng nhị phân cho mỗi danh mục:

```
>>> df_test = pd.DataFrame({"ocean_proximity": ["INLAND", "NEAR BAY"]}) >>>

pd.get_dummies(df_test)
      ocean_proximity_INLAND  ocean_proximity_NEAR BAY  0
0
1                          1  0                      1
```

Nó trông đẹp và đơn giản, vậy tại sao không sử dụng nó thay vì `OneHotEncoder`? Ưu điểm của `OneHotEncoder` là nó ghi nhớ những danh mục mà nó được huấn luyện. Điều này rất quan trọng vì khi mô hình của bạn được đưa vào sản xuất, nó cần được cung cấp chính xác các đặc trưng giống như trong quá trình huấn luyện: không hơn không kém. Hãy xem kết quả đầu ra của `cat_encoder` đã được huấn luyện khi chúng ta cho nó biến đổi cùng một `df_test` (sử dụng `transform()`, chứ không phải `fit_transform()`):

```
>>> cat_encoder.transform(df_test)
array([[0., 1., 0., 0., 0.],
       [0., 0., 0., 1., 0.]])
```

Bạn thấy sự khác biệt chứ? Hàm `get_dummies()` chỉ nhận diện hai danh mục, vì vậy nó xuất ra hai cột, trong khi `OneHotEncoder` xuất ra một cột cho mỗi danh mục đã học, theo đúng thứ tự. Hơn nữa, nếu bạn cung cấp cho `get_dummies()` một `DataFrame` chứa một danh mục chưa biết (ví dụ: "<2H OCEAN"), nó sẽ dễ dàng tạo ra một cột cho danh mục đó:

```
>>> df_test_unknown = pd.DataFrame({"ocean_proximity": ["<2H
OCEAN", "ISLAND"]})
>>> pd.get_dummies(df_test_unknown)
      ocean_proximity_<2H OCEAN  ocean_proximity_ISLAND
```

	1	0
0 1	0	1

Nhưng OneHotEncoder thông minh hơn: nó sẽ phát hiện danh mục chưa biết và gây ra một ngoại lệ. Nếu muốn, bạn có thể đặt `handle_unknown`.
 siêu tham số cần "bỏ qua", trong trường hợp đó nó sẽ chỉ đại diện cho
 Danh mục không xác định với các số không:

```
>>> cat_encoder.handle_unknown = "ignore"
>>> cat_encoder.transform(df_test_unknown)
mảng([[0., 0., 0., 0., 0.],
       [0., 0., 1., 0., 0.]])
```

Khi bạn huấn luyện bất kỳ bộ ước lượng Scikit-Learn nào bằng cách sử dụng DataFrame, bộ ước lượng đó sẽ...
 Lưu trữ tên cột trong thuộc tính `feature_names_in_`. Sau đó, Scikit-Learn đảm bảo rằng
 bất kỳ DataFrame nào được đưa vào trình ước lượng này sau đó (ví dụ:
 Hàm `to transform()` hoặc `predict()` có cùng tên cột.
 Transformer cũng cung cấp phương thức `get_feature_names_out()`.
 Bạn có thể sử dụng điều này để xây dựng một DataFrame xung quanh đầu ra của bộ chuyển đổi:

```
>>> cat_encoder.feature_names_in_
mảng(['ocean_proximity'], dtype=object)
>>> cat_encoder.get_feature_names_out()
mảng(['ocean_proximity<1H OCEAN', 'ocean_proximity_INLAND',
      'ocean_proximity_ISLAND', 'ocean_proximity_NEAR BAY',
      'ocean_proximity_NEAR OCEAN'], dtype=object)
>>> df_output =
pd.DataFrame(cat_encoder.transform(df_test_unknown),
...
columns=cat_encoder.get_feature_names_out(),
...
index=df_test_unknown.index)
...
```

MẸO

Nếu một thuộc tính phân loại có số lượng lớn các danh mục khả dĩ (ví dụ: mã quốc gia, nghề nghiệp, loài), thì mã hóa one-hot sẽ dẫn đến số lượng lớn các đặc trưng đầu vào. Điều này có thể làm chậm quá trình huấn luyện và làm giảm hiệu suất. Nếu điều này xảy ra, bạn có thể muốn thay thế đầu vào phân loại bằng các đặc trưng số hữu ích liên quan đến các danh mục: ví dụ, bạn có thể thay thế đặc trưng `ocean_proximity` bằng khoảng cách đến đại dương (tương tự, mã quốc gia có thể được thay thế bằng dân số và GDP bình quân đầu người của quốc gia đó). Ngoài ra, bạn có thể sử dụng một trong các bộ mã hóa được cung cấp bởi gói `category_encoders` trên <https://github.com/scikit-learn-contrib>. Hoặc khi làm việc với mạng nơ-ron, bạn có thể thay thế mỗi danh mục bằng một vectơ có thể học được, có chiều thấp gọi là vectơ nhúng. Đây là một ví dụ về học biểu diễn (xem Chương 13 và 17 để biết thêm chi tiết).

Điều chỉnh tỷ lệ và biến đổi tính năng

Một trong những phép biến đổi quan trọng nhất bạn cần áp dụng cho dữ liệu của mình là chuẩn hóa đặc trưng. Ngoại trừ một vài trường hợp ngoại lệ, các thuật toán Học máy không hoạt động tốt khi các thuộc tính số đầu vào có thang đo rất khác nhau.

Điều này cũng đúng với dữ liệu về nhà ở: tổng số phòng dao động từ khoảng 6 đến 39.320, trong khi thu nhập trung vị chỉ dao động từ 0 đến 15.

Nếu không có sự điều chỉnh tỷ lệ, hầu hết các mô hình sẽ có xu hướng bỏ qua thu nhập trung bình và tập trung nhiều hơn vào số lượng phòng.

Có hai cách phổ biến để làm cho tất cả các thuộc tính có cùng thang đo: sử dụng phép chia tỷ lệ min-max và chuẩn hóa.

Phương pháp chia tỷ lệ min-max (nhiều người gọi đây là chuẩn hóa) là phương pháp đơn giản nhất: đối với mỗi thuộc tính, các giá trị được dịch chuyển và chia tỷ lệ lại sao cho chúng nằm trong khoảng từ 0 đến 1. Điều này được thực hiện bằng cách trừ đi giá trị nhỏ nhất và chia cho hiệu số giữa giá trị nhỏ nhất và giá trị lớn nhất. Scikit-Learn cung cấp một bộ chuyển đổi có tên là `MinMaxScaler` cho mục đích này. Nó có một siêu tham số `feature_range` cho phép bạn thay đổi phạm vi nếu vì lý do nào đó bạn không muốn phạm vi từ 0 đến 1 (ví dụ: mạng nơ-ron hoạt động tốt nhất với đầu vào có giá trị trung bình bằng 0, vì vậy phạm vi từ -1 đến 1 là thích hợp hơn). Nó khá dễ sử dụng:

```
from sklearn.preprocessing import MinMaxScaler
```

```
min_max_scaler = MinMaxScaler(feature_range=(-1, 1))  
housing_num_min_max_scaled =  
min_max_scaler.fit_transform(housing_num)
```

CẢNH BÁO

Cũng như với tất cả các bộ ước lượng, điều quan trọng là chỉ huấn luyện các bộ chuẩn hóa trên dữ liệu huấn luyện: không bao giờ sử dụng `fit()` hoặc `fit_transform()` cho bất kỳ tập dữ liệu nào khác ngoài tập huấn luyện. Sau khi đã huấn luyện được bộ chuẩn hóa, bạn có thể sử dụng nó để `transform()` bất kỳ tập dữ liệu nào khác, bao gồm tập dữ liệu xác thực, tập dữ liệu kiểm tra và dữ liệu mới. Lưu ý rằng trong khi các giá trị trên tập huấn luyện sẽ luôn được chuẩn hóa về phạm vi được chỉ định, nếu dữ liệu mới chứa các giá trị ngoại lệ, chúng có thể bị chuẩn hóa nằm ngoài phạm vi. Nếu bạn muốn tránh điều này, chỉ cần đặt siêu tham số `clip` thành `True`.

Chuẩn hóa thì khác: trước tiên nó trừ đi giá trị trung bình (do đó các giá trị được chuẩn hóa có giá trị trung bình bằng 0), sau đó nó chia kết quả cho độ lệch chuẩn (do đó các giá trị được chuẩn hóa có độ lệch chuẩn bằng 1). Không giống như phép chia tỷ lệ min-max, chuẩn hóa không giới hạn các giá trị trong một phạm vi cụ thể. Tuy nhiên, chuẩn hóa ít bị ảnh hưởng bởi các giá trị ngoại lệ hơn.

Ví dụ, giả sử một quận có thu nhập trung vị bằng 100 (do nhầm lẫn), thay vì phạm vi thông thường là 0-15. Việc điều chỉnh tỷ lệ min-max về phạm vi 0-1 sẽ đưa giá trị ngoại lệ này xuống còn 1 và làm giảm tất cả các giá trị khác xuống phạm vi 0-0,15, trong khi việc chuẩn hóa sẽ không bị ảnh hưởng nhiều.

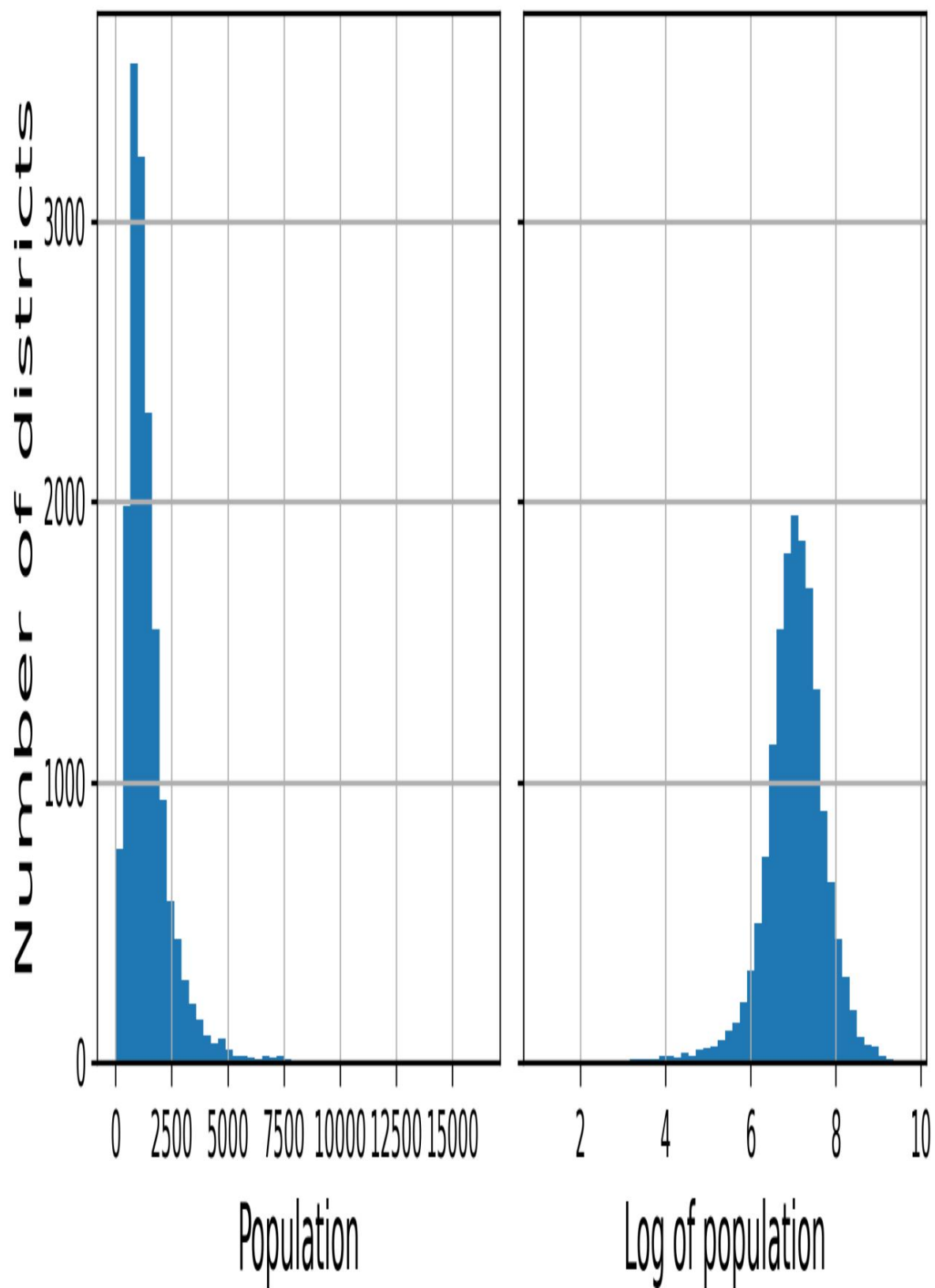
Scikit-Learn cung cấp một bộ chuyển đổi có tên là `StandardScaler` để chuẩn hóa dữ liệu:

```
from sklearn.preprocessing import StandardScaler  
  
std_scaler = StandardScaler()  
housing_num_std_scaled = std_scaler.fit_transform(housing_num)
```

MẸO

Nếu bạn muốn chuẩn hóa một ma trận thưa mà không cần chuyển đổi nó thành ma trận dày đặc trước, bạn có thể sử dụng `StandardScaler` với siêu tham số `with_mean` được đặt thành `False`: nó sẽ chỉ chia dữ liệu cho độ lệch chuẩn, mà không trừ đi giá trị trung bình (vì điều này sẽ phá vỡ tính thưa thớt).

Khi phân bố của một đặc trưng có đuôi nặng (tức là khi các giá trị cách xa giá trị trung bình không hiếm gặp theo cấp số mũ), cả phương pháp chia tỷ lệ min-max và chuẩn hóa đều sẽ thu hẹp hầu hết các giá trị vào một phạm vi nhỏ. Các mô hình Học máy nói chung không thích điều này chút nào, như chúng ta sẽ thấy trong [Chương 4](#). Vì vậy, trước khi chia tỷ lệ đặc trưng, bạn nên biến đổi nó trước để thu nhỏ đuôi nặng, và nếu có thể, làm cho phân bố gần như đối xứng. Ví dụ, một cách phổ biến để làm điều này đối với các đặc trưng dương có đuôi nặng về bên phải là thay thế đặc trưng bằng căn bậc hai của nó (hoặc nâng đặc trưng lên lũy thừa từ 0 đến 1). Nếu đặc trưng có đuôi thực sự dài và nặng, chẳng hạn như phân bố luật lũy thừa, thì việc thay thế đặc trưng bằng logarit của nó có thể hữu ích. Ví dụ, đặc trưng dân số gần như tuân theo luật lũy thừa: các quận có 10.000 dân chỉ ít xuất hiện hơn 10 lần so với các quận có 1.000 dân, chứ không phải ít xuất hiện hơn theo cấp số mũ. [Hình 2-17](#) cho thấy tính năng này trông tốt hơn nhiều khi bạn tính logarit của nó: nó rất gần với phân bố Gaussian (tức là hình chuông).



Hình 2-17. Biến đổi một đặc điểm để làm cho nó gần hơn với phân bố Gaussian.

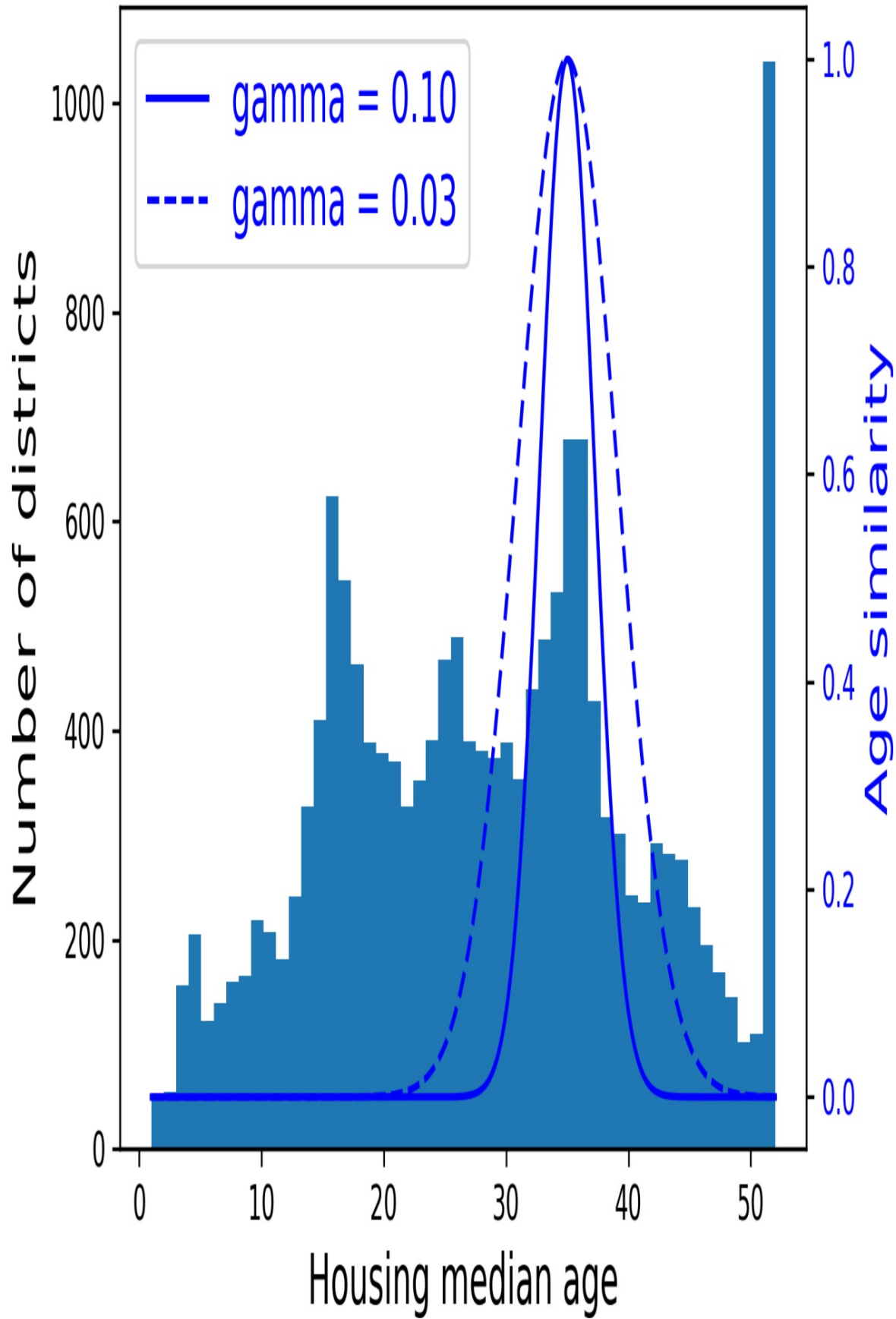
Một cách tiếp cận khác để xử lý các đặc trưng có phân bố đuôi dày là chia đặc trưng thành các nhóm (bucketizing). Điều này có nghĩa là chia phân bố của đặc trưng thành các nhóm có kích thước gần bằng nhau, và thay thế mỗi giá trị của đặc trưng bằng chỉ số của nhóm mà nó thuộc về, tương tự như cách chúng ta đã tạo ra đặc trưng `income_cat` (mặc dù chúng ta chỉ sử dụng nó cho lấy mẫu phân tầng). Ví dụ, bạn có thể thay thế mỗi giá trị bằng phần trăm của nó. Việc chia thành các nhóm có kích thước bằng nhau sẽ tạo ra một đặc trưng có phân bố gần như đồng đều, vì vậy không cần phải chuẩn hóa thêm, hoặc bạn chỉ cần chia cho số lượng nhóm để buộc các giá trị nằm trong khoảng 0-1.

Khi một đặc trưng có phân bố đa đỉnh (tức là có hai hoặc nhiều đỉnh rõ ràng, được gọi là các mode), chẳng hạn như đặc trưng `housing_median_age`, việc phân nhóm nó cũng có thể hữu ích, nhưng lần này coi các chỉ số nhóm (bucket id) là các danh mục, chứ không phải là các giá trị số. Điều này có nghĩa là các chỉ số nhóm phải được mã hóa, ví dụ như sử dụng OneHotEncoder (vì vậy bạn thường không muốn sử dụng quá nhiều nhóm). Cách tiếp cận này sẽ cho phép mô hình hồi quy dễ dàng học được các quy tắc khác nhau cho các phạm vi khác nhau của giá trị đặc trưng này. Ví dụ, có thể những ngôi nhà được xây dựng khoảng 35 năm trước có một kiểu dáng đặc biệt đã lỗi thời, và do đó chúng rẻ hơn so với tuổi đời thực tế.

Một cách tiếp cận khác để chuyển đổi các phân bố đa phương thức là thêm một đặc trưng cho mỗi phương thức (ít nhất là các phương thức chính), thể hiện sự tương đồng giữa tuổi trung vị của nhà ở và phương thức cụ thể đó. Độ đo tương đồng thường được tính toán bằng cách sử dụng Hàm Cơ sở Xuyên tâm (RBF): đây là bất kỳ hàm nào chỉ phụ thuộc vào khoảng cách giữa giá trị đầu vào và một điểm cố định. RBF được sử dụng phổ biến nhất là RBF Gaussian, có giá trị đầu ra giảm theo cấp số mũ khi giá trị đầu vào di chuyển ra xa điểm cố định. Ví dụ, độ tương đồng RBF Gaussian giữa tuổi nhà ở x và 35 được cho bởi phương trình $\exp(-(x - 35)^2)$. Siêu tham số gamma xác định tốc độ giảm của độ đo tương đồng khi x di chuyển ra xa 35. Sử dụng hàm `rbf_kernel()` của Scikit-Learn, chúng ta có thể tạo một đặc trưng RBF Gaussian mới đo độ tương đồng giữa tuổi trung vị của nhà ở và 35:


```
from sklearn.metrics.pairwise import rbf_kernel  
  
age_simil_35 = rbf_kernel(housing[["housing_median_age"]],  
                           [[35]], gamma=0.1)
```

Hình 2-18 thể hiện tính năng mới này như một hàm của tuổi trung bình của nhà ở (đường liền nét). Nó cũng cho thấy tính năng này sẽ trông như thế nào nếu chúng ta sử dụng giá trị gamma nhỏ hơn. Như biểu đồ cho thấy, tính năng tương đồng về tuổi mới đạt đỉnh điểm ở tuổi 35, ngay gần điểm nhọn trong phân bố tuổi trung bình của nhà ở: nếu nhóm tuổi cụ thể này có mối tương quan tốt với giá thấp hơn, thì rất có thể tính năng mới này sẽ hữu ích.



Hình 2-18. Đặc trưng RBF Gaussian đo lường sự tương đồng giữa tuổi trung bình của nhà ở và 35.

Cho đến nay, chúng ta mới chỉ xem xét các đặc trưng đầu vào, nhưng các giá trị mục tiêu cũng có thể cần được biến đổi. Ví dụ, nếu phân phối mục tiêu có đuôi nặng, bạn có thể chọn thay thế mục tiêu bằng logarit của nó. Nhưng nếu bạn làm vậy, mô hình hồi quy lúc này sẽ dự đoán logarit của giá trị nhà trung vị, chứ không phải chính giá trị nhà trung vị. Vì vậy, bạn cần tính lũy thừa của dự đoán của mô hình nếu muốn có giá trị nhà trung vị được dự đoán.

May mắn thay, hầu hết các bộ chuyển đổi của Scikit-Learn đều có phương thức `inverse_transform()`, giúp dễ dàng tính toán nghịch đảo của các phép biến đổi. Ví dụ, đoạn mã sau đây cho thấy cách chuẩn hóa nhãn bằng cách sử dụng `StandardScaler` (giống như chúng ta đã làm với đầu vào), sau đó huấn luyện một mô hình hồi quy tuyến tính đơn giản trên các nhãn đã được chuẩn hóa, và sử dụng nó để đưa ra dự đoán trên một số dữ liệu mới, mà chúng ta sẽ chuyển đổi trở lại tỷ lệ ban đầu bằng cách sử dụng phương thức `inverse_transform()` của bộ chuẩn hóa đã được huấn luyện. Lưu ý rằng chúng ta chuyển đổi các nhãn từ Pandas Series sang DataFrame, vì `StandardScaler` yêu cầu đầu vào 2D. Ngoài ra, trong ví dụ này, chúng ta chỉ huấn luyện mô hình trên một đặc trưng đầu vào thô duy nhất (thu nhập trung vị), để đơn giản hóa.

```
from sklearn.linear_model import LinearRegression

target_scaler = StandardScaler() scaled_labels =
target_scaler.fit_transform(housing_labels.to_frame())

model = LinearRegression()
model.fit(housing[["median_income"]], scaled_labels) some_new_data =
housing[["median_income"]].iloc[:5] # giả vờ đây là dữ liệu mới

scaled_predictions = model.predict(some_new_data) predictions =
target_scaler.inverse_transform(scaled_predictions)
```

Cách này hoạt động tốt, nhưng có một cách đơn giản hơn: chúng ta có thể sử dụng `TransformedTargetRegressor`. Chúng ta chỉ cần tạo nó, cung cấp cho nó mô hình hồi quy và bộ chuyển đổi nhãn, sau đó huấn luyện nó trên tập dữ liệu huấn luyện.

Tập dữ liệu được thiết lập bằng cách sử dụng các nhãn gốc chưa được hiệu chỉnh. Nó sẽ tự động sử dụng bộ chuyển đổi (transformer) để hiệu chỉnh các nhãn và huấn luyện mô hình hồi quy trên các nhãn đã được hiệu chỉnh, giống như chúng ta đã làm ở trên. Sau đó, khi chúng ta đưa ra dự đoán, nó sẽ gọi phương thức `predict()` của mô hình hồi quy và sử dụng phương thức `inverse_transform()` của bộ hiệu chỉnh để tạo ra các dự đoán:

```
from sklearn.compose import TransformedTargetRegressor

model = TransformedTargetRegressor(LinearRegression(),
                                   transformer=StandardScaler())
model.fit(housing[["median_income"]], housing_labels)
predictions = model.predict(some_new_data)
```

Máy biến áp tùy chỉnh

Mặc dù Scikit-Learn cung cấp nhiều bộ chuyển đổi hữu ích, bạn vẫn cần tự viết các bộ chuyển đổi của riêng mình cho các tác vụ như chuyển đổi tùy chỉnh, các thao tác làm sạch hoặc kết hợp các thuộc tính cụ thể.

Đối với các phép biến đổi không yêu cầu huấn luyện, bạn chỉ cần viết một hàm nhận mảng NumPy làm đầu vào và xuất ra mảng đã được biến đổi. Ví dụ, như chúng ta đã thảo luận trong phần trước, thường nên biến đổi các đặc trưng có phân bố đuôi nặng bằng cách thay thế chúng bằng logarit của chúng (giả sử đặc trưng đó dương và đuôi nằm bên phải). Hãy tạo một bộ biến đổi logarit và áp dụng nó cho đặc trưng dân số:

```
from sklearn.preprocessing import FunctionTransformer

log_transformer = FunctionTransformer(np.log,
                                     inverse_func=np.exp)
log_pop = log_transformer.transform(housing[["population"]])
```

Tham số `inverse_func` là tùy chọn. Nó cho phép bạn chỉ định một hàm biến đổi nghịch đảo, ví dụ như nếu bạn dự định sử dụng bộ chuyển đổi của mình trong `TransformedTargetRegressor`.

Hàm biến đổi của bạn có thể nhận các siêu tham số làm đối số bổ sung. Ví dụ, đây là cách tạo một bộ biến đổi tính toán...

Cùng một thước đo độ tương đồng RBF Gaussian như trước đây:

```
rbf_transformer = FunctionTransformer(rbf_kernel,
                                     kw_args=dict(Y=[[35.]]),
                                     gamma=0.1)
age_simil_35 =
rbf_transformer.transform(housing[["housing_median_age"]])
```

Lưu ý rằng không có hàm nghịch đảo cho nhân RBF, vì luôn có hai giá trị tại một khoảng cách nhất định từ một điểm cố định (ngoại trừ tại khoảng cách 0). Cũng cần lưu ý rằng hàm `rbf_kernel()` không xử lý các đặc trưng một cách riêng biệt. Nếu bạn truyền vào một mảng gồm 2 đặc trưng, nó sẽ đo khoảng cách 2D (Euclidean) để đo độ tương đồng. Ví dụ, đây là cách thêm một đặc trưng để đo độ tương đồng về địa lý giữa mỗi quận và thành phố San Francisco:

```
sf_coords = 37.7749, -122.41
sf_transformer = FunctionTransformer(rbf_kernel,
                                     kw_args=dict(Y=[sf_coords],
                                     gamma=0.1))
sf_simil = sf_transformer.transform(housing[["latitude",
"longitude"]])
```

Các bộ chuyển đổi tùy chỉnh cũng rất hữu ích để kết hợp các tính năng. Ví dụ, đây là một `FunctionTransformer` tính toán tỷ lệ giữa các tính năng đầu vào 0 và 1:

```
>>> ratio_transformer = FunctionTransformer(lambda X: X[:, [0]] / X[:,
[1]]) >>>
ratio_transformer.transform(np.array([[1., 2.], [3., 4.])))
array([[0.5 ],
       [0.75]])
```

`FunctionTransformer` rất tiện dụng, nhưng nếu bạn muốn transformer của mình có thể huấn luyện được, học một số tham số trong phương thức `fit()` và sử dụng chúng sau này trong phương thức `transform()`? Để làm được điều này, bạn cần viết một lớp tùy chỉnh. Scikit-Learn dựa trên kiểu dữ liệu linh hoạt (duck typing), vì vậy lớp này không cần phải kế thừa từ bất kỳ lớp cơ sở cụ thể nào: tất cả những gì nó cần là ba tham số.

các phương thức: `fit()` (phải trả về chính nó), `transform()` và `fit_transform()`.

Bạn có thể sử dụng phương thức `fit_transform()` miễn phí bằng cách thêm `TransformerMixin` làm lớp cơ sở: cách triển khai mặc định sẽ chỉ gọi `fit()` rồi `transform()`. Nếu bạn thêm `BaseEstimator` làm lớp cơ sở (và tránh sử dụng `*args` và `**kwargs` trong hàm tạo), bạn cũng sẽ nhận được thêm hai phương thức: `get_params()` và `set_params()`.

Những công cụ này sẽ hữu ích cho việc điều chỉnh siêu tham số tự động.

Ví dụ, đây là một máy biến áp tùy chỉnh hoạt động tương tự như...
`StandardScaler`:

```
from sklearn.base import BaseEstimator, TransformerMixin from
sklearn.utils.validation import check_array, check_is_fitted

class StandardScalerClone(BaseEstimator, TransformerMixin):
    def __init__(self, with_mean=True): # không có *args hoặc **kwargs! self.with_mean =
        with_mean

    def fit(self, X, y=None): # y là bắt buộc mặc dù chúng ta
        # đừng sử dụng nó
        X = check_array(X) # kiểm tra xem X có phải là một mảng với
        # giá trị float hữu hạn self.mean_
        = X.mean(axis=0) self.scale_ = X.std(axis=0)
        self.n_features_in_ = X.shape[1] # mỗi bộ
        # ước lượng lưu trữ điều này trong fit()

        # trả về chính nó # luôn luôn trả về chính nó!

    def transform(self, X):
        check_is_fitted(self) # tìm kiếm các thuộc tính đã học (với dấu gạch dưới
        # phía sau)
        X = check_array(X) assert
        self.n_features_in_ == X.shape[1] if self.with_mean: X =
        X - self.mean_ return X /
        self.scale_
```

Dưới đây là một vài điều cần lưu ý:

- Gói `sklearn.utils.validation` chứa một số hàm mà chúng ta có thể sử dụng để kiểm tra tính hợp lệ của dữ liệu đầu vào. Để đơn giản, chúng ta sẽ bỏ qua các bài kiểm tra này trong phần còn lại của cuốn sách, nhưng mã sản xuất cần phải có chúng.
- Các pipeline của Scikit-Learn yêu cầu phương thức `fit()` phải có hai tham số `X` và `y`, đó là lý do tại sao chúng ta cần tham số `y=None` ngay cả khi chúng ta không sử dụng `y`.
- Tất cả các bộ ước lượng của Scikit-Learn đều thiết lập `n_features_in_` trong phương thức `fit()`, và chúng đảm bảo rằng dữ liệu được truyền đến `transform()` hoặc `predict()` có số lượng đặc trưng này.
- Phương thức `fit()` phải trả về chính nó.
- Việc triển khai này chưa hoàn thiện 100%: tất cả các bộ ước lượng cần phải thiết lập `'feature_names_in_'` trong phương thức `'fit()'` khi chúng được truyền vào một `'DataFrame'`. Hơn nữa, tất cả các bộ chuyển đổi cần phải cung cấp phương thức `'get_feature_names_out()'`, cũng như phương thức `'inverse_transform()'` khi phép biến đổi của chúng có thể đảo ngược. Xem bài tập cuối cùng ở cuối chương này để biết thêm chi tiết.

MẸO

Bạn có thể kiểm tra xem trình ước lượng tùy chỉnh của mình có tuân thủ API của Scikit-Learn hay không bằng cách truyền một thể hiện vào hàm `check_estimator()` từ gói `sklearn.utils.estimator_checks`. Để xem API đầy đủ, hãy truy cập <https://scikit-learn.org/stable/developers/>.

Một bộ chuyển đổi tùy chỉnh có thể (và thường là vậy) sử dụng các bộ ước lượng khác trong quá trình triển khai. Ví dụ, bên dưới chúng ta thấy một bộ chuyển đổi tùy chỉnh sử dụng bộ phân cụm KMeans trong phương thức `fit()` để xác định các cụm chính trong dữ liệu huấn luyện, và sau đó sử dụng `rbf_kernel()` trong phương thức `transform()` để đo lường mức độ tương đồng của mỗi mẫu với từng tâm cụm:

```

from sklearn.cluster import KMeans

class ClusterSimilarity(BaseEstimator, TransformerMixin):
    def __init__(self, n_clusters=10, gamma=1.0, random_state=None):

        self.n_clusters = n_clusters
        self.gamma = gamma
        self.random_state = random_state

    def fit(self, X, y=None, sample_weight=None):
        self.kmeans_ = KMeans(self.n_clusters,
                                random_state=self.random_state)
        self.kmeans_.fit(X, sample_weight=sample_weight)
        return self # luôn trả về self!

    def transform(self, X):
        return rbf_kernel(X, self.kmeans_.cluster_centers_, gamma=self.gamma)

    def get_feature_names_out(self, names=None):
        trả về [f"Độ tương đồng của cụm {i} " cho i trong
        phạm vi(self.n_clusters)]

```

Như chúng ta sẽ thấy trong **Chương 9**, K-Means là một thuật toán phân cụm giúp xác định các cụm trong dữ liệu. Số lượng cụm cần tìm được điều khiển bởi siêu tham số `n_clusters`. Sau khi huấn luyện, tâm của các cụm có sẵn thông qua thuộc tính `cluster_centers_`. Phương thức `fit()` của `KMeans` hỗ trợ một đối số tùy chọn `sample_weight` cho phép người dùng chỉ định trọng số tương đối của các mẫu. K-Means là một thuật toán ngẫu nhiên, có nghĩa là nó dựa vào tính ngẫu nhiên để xác định các cụm, vì vậy nếu bạn muốn có kết quả có thể tái tạo, bạn phải đặt tham số `random_state`. Như bạn thấy, mặc dù nhiệm vụ khá phức tạp, mã nguồn lại khá đơn giản. Bây giờ chúng ta hãy sử dụng bộ chuyển đổi tùy chỉnh này:

```

cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
similarities =
cluster_simil.fit_transform(housing[["latitude", "longitude"]],

sample_weight=housing_labels)

```


Đoạn mã này tạo ra một bộ chuyển đổi ClusterSimilarity, thiết lập...

tăng số cụm lên 10 cụm. Sau đó, nó gọi hàm fit_transform() với

vĩ độ và kinh độ của mỗi quận trong tập dữ liệu huấn luyện, trọng số

mỗi quận được xác định theo giá trị nhà trung bình. Thuật toán chuyển đổi sử dụng K-Means để...

Xác định các cụm, sau đó đo độ tương đồng RBF Gaussian giữa chúng.

Mỗi quận và tất cả 10 trung tâm cụm. Kết quả là một ma trận với một hàng.

Mỗi quận/huyện có một cột tương ứng với một cụm. Chúng ta hãy xem xét 3 hàng đầu tiên.

Làm tròn đến 2 chữ số thập phân:

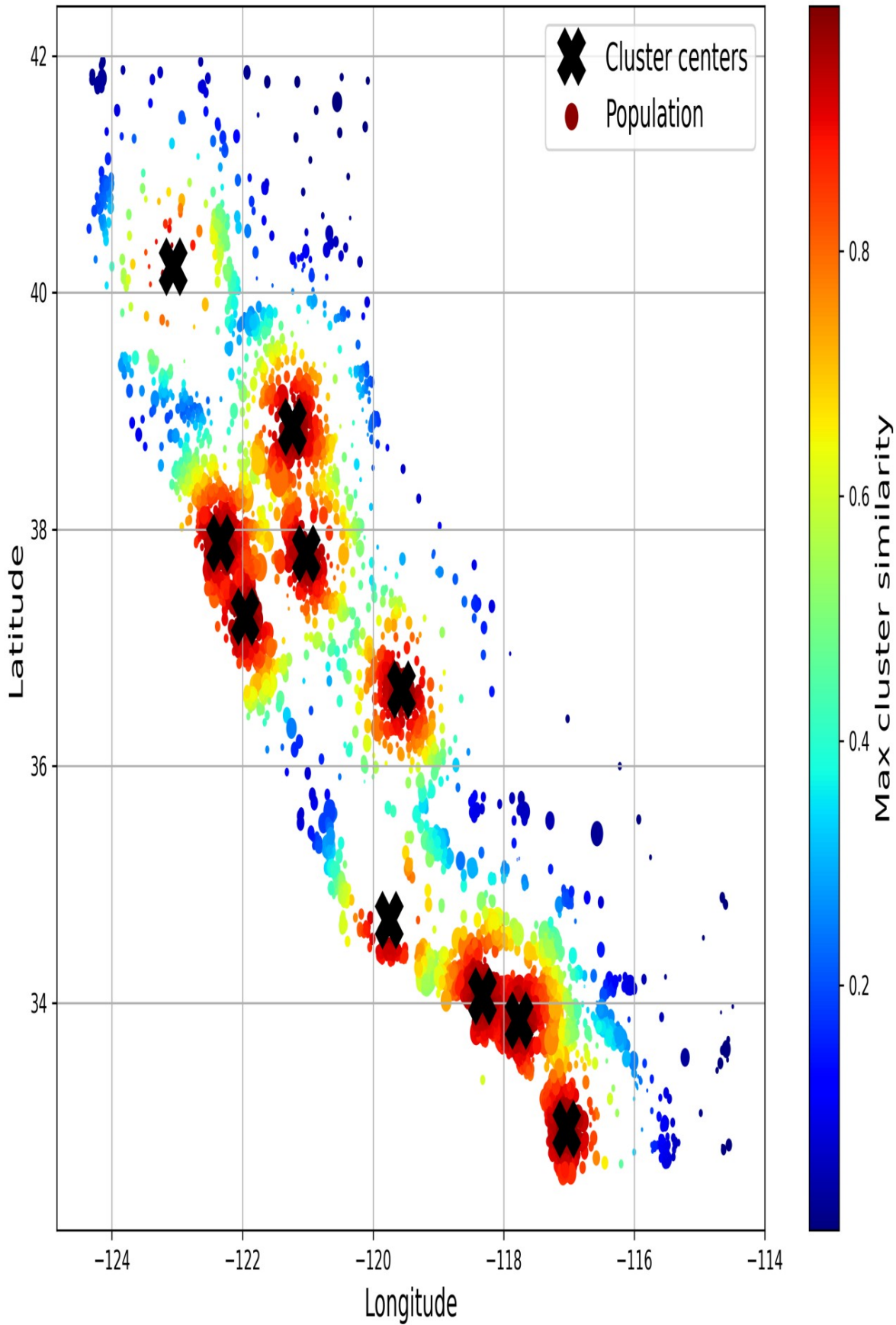
```
>>> similarities[:3].round(2)
mảng([[0. , 0.14, 0. ],      , 0. , 0. , 0,08, 0. , 0,99, 0. , 0,6
      [0,63, 0. , 0,99, 0. , 0. , 0. , 0,04, 0. , 0.11, 0.
],
      [0. , 0.29, 0. , 0. , 0.01, 0.44, 0. , 0,7 , 0. , 0,3
]])
```

Hình 2-19 hiển thị 10 trung tâm cụm được tìm thấy bằng thuật toán K-Means. Các quận

được tô màu theo sự tương đồng về địa lý với cụm gần nhất của chúng.

trung tâm. Như bạn thấy, hầu hết các cụm đều nằm ở những khu vực đông dân cư và

những khu vực đất đỏ.



Hình 2-19. Độ tương đồng RBF Gaussian với tâm cụm gần nhất.

Các đường ống chuyển đổi

Như bạn thấy, có rất nhiều bước chuyển đổi dữ liệu cần được thực hiện theo đúng thứ tự. May mắn thay, Scikit-Learn cung cấp lớp Pipeline để hỗ trợ các chuỗi chuyển đổi như vậy. Dưới đây là một pipeline nhỏ dành cho các thuộc tính số, trong đó bước đầu tiên sẽ điền giá trị thiếu rồi đến bước chuẩn hóa các đặc trưng đầu vào:

```
from sklearn.pipeline import Pipeline

num_pipeline = Pipeline([
    ("impute", SimpleImputer(strategy="median")), ("standardize",
    StandardScaler()),
])
```

Hàm tạo Pipeline nhận một danh sách các cặp tên/bộ ước lượng (bộ 2 phần tử) xác định một chuỗi các bước. Tên có thể là bất kỳ thứ gì bạn muốn, miễn là chúng duy nhất và không chứa dấu gạch dưới kép, __. Chúng sẽ hữu ích sau này khi chúng ta thảo luận về việc điều chỉnh siêu tham số. Tất cả các bộ ước lượng phải là bộ chuyển đổi (nghĩa là chúng phải có phương thức fit_transform()), ngoại trừ bộ ước lượng cuối cùng, có thể là bất kỳ thứ gì: bộ chuyển đổi, bộ dự đoán hoặc bất kỳ loại bộ ước lượng nào khác.

MẸO

Trong Jupyter notebook, nếu bạn nhập sklearn và chạy `sklearn.set_config(display="diagram")`, tất cả các thuật toán ước lượng của Scikit-Learn sẽ được hiển thị dưới dạng sơ đồ tương tác. Điều này đặc biệt hữu ích cho việc trực quan hóa các pipeline. Để trực quan hóa num_pipeline, hãy chạy một ô với num_pipeline là dòng cuối cùng. Nhấp vào một thuật toán ước lượng sẽ hiển thị thêm chi tiết.

Nếu bạn không muốn mất công đặt tên cho các transformer, bạn có thể sử dụng hàm make_pipeline() thay thế: hàm này chỉ nhận các transformer làm đối số vị trí và tạo ra một Pipeline sử dụng tên của chúng.

Tên các lớp của bộ chuyển đổi, viết thường và không có dấu gạch dưới (ví dụ: "simpleimputer"). Trong trường hợp nhiều bộ chuyển đổi có cùng tên, một chỉ số sẽ được thêm vào tên của chúng (ví dụ: "foo-1", "foo-2", v.v.).

```
from sklearn.pipeline import make_pipeline

num_pipeline = make_pipeline(SimpleImputer(strategy="median"),
                             StandardScaler())
```

Khi bạn gọi phương thức `fit()` của pipeline, nó sẽ gọi `fit_transform()` tuần tự trên tất cả các transformer, truyền kết quả của mỗi lần gọi làm tham số cho lần gọi tiếp theo cho đến khi đạt đến estimator cuối cùng, lúc đó nó chỉ gọi phương thức `fit()`.

Đường dẫn xử lý (pipeline) cung cấp các phương thức tương tự như bộ ước lượng cuối cùng. Trong ví dụ này, bộ ước lượng cuối cùng là `StandardScaler`, vốn là một bộ chuyển đổi (transformer), vì vậy đường dẫn xử lý cũng hoạt động như một bộ chuyển đổi. Nếu bạn gọi phương thức `transform()` của đường dẫn xử lý, nó sẽ áp dụng tuần tự tất cả các phép biến đổi cho dữ liệu. Nếu bộ ước lượng cuối cùng là một bộ dự đoán (predictor) thay vì một bộ chuyển đổi, thì đường dẫn xử lý sẽ có phương thức `predict()` thay vì phương thức `transform()`. Việc gọi phương thức này sẽ áp dụng tuần tự tất cả các phép biến đổi cho dữ liệu và truyền kết quả cho phương thức `predict()` của bộ dự đoán.

Hãy gọi phương thức `fit_transform()` của pipeline và xem hai hàng đầu tiên của kết quả, được làm tròn đến 2 chữ số thập phân:

```
>>> housing_num_prepared =
num_pipeline.fit_transform(housing_num) >>>
housing_num_prepared[:2].round(2) array([[ -1.42,
  1.01,  1.86,  0.31,  1.37,  0.14,  1.39, -0.94],
 [ 0.6 , -0.7 ,  1.17]])
```

Như chúng ta đã thấy trước đó, nếu bạn muốn khôi phục một DataFrame tốt, bạn có thể sử dụng phương thức `get_feature_names_out()` của pipeline:

```
df_housing_num_prepared =
    pd.DataFrame( housing_num_prepared,
```

```
columns=num_pipeline.get_feature_names_out(),
index=housing_num.index)
```

Các pipeline hỗ trợ lập chỉ mục, ví dụ pipeline[1] trả về bộ ước lượng thứ hai trong pipeline, và pipeline[:-1] trả về một đối tượng Pipeline chứa tất cả các bộ ước lượng ngoại trừ bộ ước lượng cuối cùng. Bạn cũng có thể truy cập các bộ ước lượng thông qua thuộc tính steps, là một danh sách các cặp tên/bộ ước lượng, hoặc thông qua thuộc tính từ điển name_steps, ánh xạ các tên đến các bộ ước lượng. Ví dụ num_pipeline["simpleimputer"] trả về bộ ước lượng có tên "simpleimputer".

Cho đến nay, chúng ta đã xử lý các cột phân loại và các cột số một cách riêng biệt. Sẽ thuận tiện hơn nếu có một bộ chuyển đổi duy nhất có khả năng xử lý tất cả các cột, áp dụng các phép biến đổi phù hợp cho từng cột. Để làm được điều này, bạn có thể sử dụng ColumnTransformer. Ví dụ, ColumnTransformer sau đây sẽ áp dụng num_pipeline (cái mà chúng ta vừa định nghĩa) cho các thuộc tính số và cat_pipeline cho thuộc tính phân loại:

```
from sklearn.compose import ColumnTransformer

num_attribs = ["longitude", "latitude", "housing_median_age", "total_rooms",
               ["tổng_số_phòng_ngủ", "dân_lượng", "hộ_hộ",
               "thu_nhu_lượng_trung_bình"]] cat_attribs = ["khoảng_xa_biển"]

cat_pipeline = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore"))

tiền_xử_ly = ColumnTransformer([
    ("num", num_pipeline, num_attribs), ("cat",
    cat_pipeline, cat_attribs),
])
```

Đầu tiên, chúng ta nhập lớp ColumnTransformer, tiếp theo, chúng ta định nghĩa danh sách tên cột số và cột phân loại, xây dựng một pipeline đơn giản cho các thuộc tính phân loại, và cuối cùng, chúng ta xây dựng một...

ColumnTransformer. Hàm tạo của nó yêu cầu một danh sách các bộ ba (bộ 3 phần tử), mỗi bộ ba chứa một tên (phải là duy nhất và không chứa dấu gạch dưới kép), một bộ chuyển đổi, và một danh sách các tên (hoặc chỉ số) của các cột mà bộ chuyển đổi đó sẽ được áp dụng.

MẸO

Thay vì sử dụng bộ chuyển đổi, bạn có thể chỉ định chuỗi "drop" nếu muốn loại bỏ các cột, hoặc bạn có thể chỉ định "passthrough" nếu muốn giữ nguyên các cột. Theo mặc định, các cột còn lại (tức là những cột không được liệt kê) sẽ bị loại bỏ, nhưng bạn có thể đặt siêu tham số `remainder` thành bất kỳ bộ chuyển đổi nào (hoặc thành "passthrough") nếu muốn xử lý các cột này theo cách khác.

Vì việc liệt kê tất cả tên cột không thực sự tiện lợi, Scikit-Learn cung cấp hàm `make_column_selector()` trả về một hàm chọn mà bạn có thể sử dụng để tự động chọn tất cả các đặc trưng thuộc một loại nhất định, chẳng hạn như số hoặc phân loại. Hàm chọn này có thể được truyền cho `ColumnTransformer` thay vì tên cột hoặc chỉ số. Hơn nữa, nếu bạn không quan tâm đến việc đặt tên cho các bộ chuyển đổi, bạn có thể sử dụng `make_column_transformer()` để chọn tên cho bạn, giống như `make_pipeline()`. Ví dụ, đoạn mã sau tạo ra cùng một `ColumnTransformer` như trên, ngoại trừ các bộ chuyển đổi được tự động đặt tên là "pipeline-1" và "pipeline-2" thay vì "num" và "cat":

```
from sklearn.compose import make_column_selector, make_column_transformer

tiền xử lý = make_column_transformer( (num_pipeline,

make_column_selector(dtype_include=np.number)), (cat_pipeline,

make_column_selector(dtype_include=np.object)), )
```

Giờ chúng ta đã sẵn sàng áp dụng ColumnTransformer này vào dữ liệu nhà ở:

```
housing_prepared = preprocessing.fit_transform(housing)
```

Tuyệt vời! Chúng ta có một quy trình tiền xử lý lấy toàn bộ dữ liệu huấn luyện và áp dụng từng bộ chuyển đổi cho các cột phù hợp, sau đó nối các cột đã được chuyển đổi theo chiều ngang (các bộ chuyển đổi không bao giờ được thay đổi số hàng). Một lần nữa, điều này trả về một mảng NumPy, nhưng bạn có thể lấy tên cột bằng cách sử dụng `preprocessing.get_feature_names_out()` và đóng gói dữ liệu vào một DataFrame đẹp mắt như chúng ta đã làm trước đây.

GHI CHÚ

OneHotEncoder trả về ma trận thưa, trong khi `num_pipeline` trả về ma trận dày. Khi có sự kết hợp giữa ma trận thưa và ma trận dày, ColumnTransformer sẽ ước tính mật độ của ma trận cuối cùng (tức là tỷ lệ các ô khác 0), và nó trả về ma trận thưa nếu mật độ thấp hơn một ngưỡng nhất định (mặc định là `sparse_threshold=0.3`). Trong ví dụ này, nó trả về ma trận dày.

Dự án của bạn đang tiến triển rất tốt, bạn gần như đã sẵn sàng để huấn luyện một số mô hình rồi! Giờ bạn muốn tạo một pipeline duy nhất thực hiện tất cả các phép biến đổi mà bạn đã thử nghiệm cho đến nay. Hãy cùng tóm tắt lại pipeline này sẽ làm gì và tại sao:

- Các giá trị thiếu trong các đặc trưng số sẽ được thay thế bằng giá trị trung vị, vì hầu hết các thuật toán học máy không lường trước được các giá trị thiếu. Đối với đặc trưng phân loại, bất kỳ giá trị thiếu nào sẽ được thay thế bằng danh mục xuất hiện thường xuyên nhất.
- Đặc trưng phân loại sẽ được mã hóa one-hot, vì hầu hết các thuật toán học máy chỉ chấp nhận đầu vào dạng số.

- Một vài đặc điểm tỷ lệ sẽ được tính toán và thêm vào: tỷ lệ phòng ngủ, số phòng trên mỗi nhà và số người trên mỗi nhà. Hy vọng những đặc điểm này sẽ tương quan tốt hơn với giá trị nhà ở trung bình, và do đó giúp ích cho các mô hình học máy.
- Một vài đặc điểm về sự tương đồng giữa các cụm cũng sẽ được thêm vào. Những đặc điểm này có thể sẽ hữu ích hơn cho mô hình so với vĩ độ và kinh độ.
- Các đặc trưng có đuôi dài sẽ được thay thế bằng logarit của chúng, vì hầu hết các mô hình đều ưu tiên các đặc trưng có phân bố gần như đồng đều hoặc phân bố Gauss.
- Tất cả các đặc trưng số sẽ được chuẩn hóa, vì hầu hết các thuật toán học máy đều ưu tiên khi tất cả các đặc trưng có thang đo gần như nhau.

Đến giờ chắc hẳn bạn đã quen thuộc với đoạn mã xây dựng quy trình thực hiện tất cả những điều này rồi:

```
def column_ratio(X):
    return X[:, [0]] / X[:, [1]]

def ratio_pipeline(name=None): return
    make_pipeline(
        SimpleImputer(strategy="median"),
        FunctionTransformer(column_ratio,
                             feature_names_out=lambda
input_features: [name]),
        StandardScaler())

log_pipeline = make_pipeline(SimpleImputer(strategy="median"),
                             FunctionTransformer(np.log),
                             StandardScaler())

cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
default_num_pipeline
=
make_pipeline(SimpleImputer(strategy="median"), StandardScaler())

tiền xử lý = ColumnTransformer([ ("bedrooms_ratio",
    ratio_pipeline("bedrooms_ratio"),
    ["tổng_phòng_ngủ", "tổng_phòng_sống"]),
    ("phòng_mỗi_nhà", ratio_pipeline("phòng_mỗi_nhà"), ["tổng_phòng",
    "hộ_cư"]), ("người_mỗi_nhà",
    ratio_pipeline("người_mỗi_nhà"), ["dân_cư", "hộ_cư"])]
```



```

        ("log", log_pipeline, ["total_bedrooms", "total_rooms",
                               "dân số", "hộ gia đình",
                               "median_income"]),
        ("geo", cluster_simil, ["latitude", "longitude"]), ("cat",
        cat_pipeline,
        make_column_selector(dtype_include=np.object)),

    ], remainder=default_num_pipeline) # còn lại một cột: housing_median_age

```

Nếu bạn chạy bộ chuyển đổi cột này, nó sẽ thực hiện tất cả các phép biến đổi và xuất ra một mảng NumPy với 24 đặc trưng:

```

>>> housing_prepared = preprocessing.fit_transform(housing) >>>
housing_prepared.shape (16512, 24)
>>>
preprocessing.get_feature_names_out()
array(['bedrooms_ratio__bedrooms_ratio',
       'rooms_per_house__rooms_per_house',
       'people_per_house__people_per_house',
       'log__total_bedrooms',
       'log__total_rooms', 'log__population', 'log__households',
       'log__median_income', 'geo__Cluster 0 similarity', [...], 'geo__Cluster 9
       similarity', 'cat__ocean_proximity_<1H OCEAN',

       'cat__ocean_proximity_INLAND',
       'cat__ocean_proximity_ISLAND',
       'cat__ocean_proximity_NEAR BAY',
       'cat__ocean_proximity_NEAR OCEAN',
       'remainder__housing_median_age'], dtype=object)

```

Chọn và huấn luyện một mô hình

Cuối cùng! Bạn đã xác định được vấn đề, thu thập dữ liệu và phân tích chúng, lấy mẫu tập huấn luyện và tập kiểm tra, và viết một quy trình tiền xử lý để tự động làm sạch và chuẩn bị dữ liệu cho các thuật toán Học máy. Giờ đây, bạn đã sẵn sàng lựa chọn và huấn luyện một mô hình Học máy.

Huấn luyện và đánh giá trên bộ dữ liệu huấn luyện

Tin vui là nhờ tất cả các bước trước đó, mọi việc giờ đây sẽ trở nên dễ dàng! Bạn quyết định huấn luyện một mô hình Hồi quy tuyến tính rất cơ bản để bắt đầu:

```
from sklearn.linear_model import LinearRegression

lin_reg = make_pipeline(preprocessing, LinearRegression()) lin_reg.fit(housing,
housing_labels)
```

Xong! Giờ bạn đã có một mô hình Hồi quy tuyến tính hoạt động. Hãy thử nghiệm nó trên tập dữ liệu huấn luyện, xem xét 5 dự đoán đầu tiên và so sánh chúng với nhãn:

```
>>> housing_predictions = lin_reg.predict(housing) >>> housing_predictions[:5].round(-2)
# -2 = làm tròn đến hàng trăm gần nhất array([243700., 372400., 128800., 94400., 328300.]) >>>
housing_labels.iloc[:5].values
array([458300., 483800., 101700., 96100., 361800.])
```

Vâng, nó hoạt động, nhưng không phải lúc nào cũng vậy: dự đoán đầu tiên sai lệch rất nhiều (hơn 200.000 đô la!), trong khi các dự đoán khác tốt hơn: hai dự đoán sai lệch khoảng 25%, và hai dự đoán sai lệch dưới 10%. Hãy nhớ rằng bạn đã chọn sử dụng RMSE làm thước đo hiệu suất, vì vậy bạn muốn đo RMSE của mô hình hồi quy này trên toàn bộ tập dữ liệu huấn luyện bằng cách sử dụng hàm `mean_squared_error()` của Scikit-Learn, với đối số `squared` được đặt thành `False`:

```
>>> from sklearn.metrics import mean_squared_error >>> lin_rmse =
mean_squared_error(housing_labels, housing_predictions,

...                               bình phương=Sai)
...
>>> lin_rmse
68687.89176589991
```

Kết quả này tuy tốt hơn không có gì, nhưng rõ ràng không phải là một điểm số tuyệt vời: giá trị nhà ở trung bình của hầu hết các quận nằm trong khoảng từ 120.000 đến 265.000 đô la, vì vậy sai số dự đoán điển hình là 68.628 đô la thực sự không mấy khả quan. Đây là một ví dụ về mô hình không phù hợp với dữ liệu huấn luyện. Khi điều này xảy ra, nó có thể có nghĩa là các đặc trưng không cung cấp đủ thông tin để đưa ra dự đoán tốt, hoặc mô hình không đủ mạnh. Như chúng ta đã thấy trong chương trước, các cách chính để khắc phục tình trạng không phù hợp là chọn một mô hình mạnh hơn, cung cấp cho thuật toán huấn luyện các đặc trưng tốt hơn, hoặc giảm bớt các ràng buộc trên mô hình. Mô hình này không được điều chỉnh (regularized), điều này loại trừ lựa chọn cuối cùng. Bạn có thể thử thêm nhiều đặc trưng hơn, nhưng trước tiên bạn nên thử một mô hình phức tạp hơn để xem hiệu quả của nó như thế nào.

Vì vậy, bạn quyết định thử sử dụng `DecisionTreeRegressor`, vì đây là một mô hình khá mạnh mẽ có khả năng tìm ra các mối quan hệ phi tuyến tính phức tạp trong dữ liệu (Cây quyết định được trình bày chi tiết hơn trong [Chương 6](#)):

```
from sklearn.tree import DecisionTreeRegressor

tree_reg = make_pipeline(preprocessing,
                          DecisionTreeRegressor(random_state=42))
tree_reg.fit(housing, housing_labels)
```

Sau khi mô hình đã được huấn luyện, bạn sẽ đánh giá nó trên tập dữ liệu huấn luyện:

```
>>> housing_predictions = tree_reg.predict(housing) >>> tree_rmse
= mean_squared_error(housing_labels, housing_predictions,
...                  bình phương=Sai)
...
>>> tree_rmse 0.0
```

Khoan đã, cái gì vậy!? Không hề có lỗi nào sao? Liệu mô hình này thực sự hoàn hảo đến vậy? Tất nhiên, khả năng cao hơn là mô hình đã bị quá khớp dữ liệu một cách nghiêm trọng. Làm sao bạn có thể chắc chắn? Như chúng ta đã thấy trước đó, bạn không nên động đến tập dữ liệu kiểm thử cho đến khi bạn sẵn sàng triển khai một mô hình mà bạn tự tin, vì vậy bạn cần sử dụng một phần tập dữ liệu huấn luyện để huấn luyện và một phần để xác thực mô hình.

Đánh giá tốt hơn bằng phương pháp kiểm chứng chéo

Một cách để đánh giá mô hình Cây quyết định là sử dụng hàm `train_test_split()` để chia tập dữ liệu huấn luyện thành một tập dữ liệu huấn luyện nhỏ hơn và một tập dữ liệu kiểm chứng, sau đó huấn luyện mô hình của bạn trên tập dữ liệu huấn luyện nhỏ hơn và đánh giá chúng trên tập dữ liệu kiểm chứng. Việc này tốn một chút công sức, nhưng không quá khó khăn, và nó sẽ hoạt động khá tốt.

Một giải pháp thay thế tuyệt vời là sử dụng tính năng kiểm định chéo K-fold của Scikit-Learn . Đoạn mã sau chia ngẫu nhiên tập dữ liệu huấn luyện thành 10 tập con không chồng chéo nhau gọi là fold, sau đó huấn luyện và đánh giá mô hình Cây quyết định 10 lần, mỗi lần chọn một fold khác nhau để đánh giá và sử dụng 9 fold còn lại để huấn luyện. Kết quả là một mảng chứa 10 kết quả đánh giá.

điểm số:

```
from sklearn.model_selection import cross_val_score

tree_rmses = -cross_val_score(tree_reg, housing, housing_labels,
                               scoring="neg_root_mean_squared_error", cv=10)
```

CẢNH BÁO

Các đặc trưng kiểm định chéo của Scikit-Learn mong đợi một hàm tiện ích (càng lớn càng tốt) chứ không phải một hàm chi phí (càng nhỏ càng tốt), vì vậy hàm tính điểm thực chất là ngược lại với RMSE. Đó là một giá trị âm, đó là lý do tại sao chúng ta cần đảo dấu của đầu ra để có được điểm RMSE.

Hãy cùng xem kết quả:

```
>>> pd.Series(tree_rmses).describe() count
10.000000
nghĩa là      66868.027288
độ lệch      2060.966425
chuẩn      63649.536493
tối thiểu 25% 65338.078316
50%          66801.953094
75%          68229.934454
```

kiểu

70094.778246

dữ liệu tối đa: float64

Giờ đây, cây quyết định trông không còn tốt như trước nữa. Trên thực tế, nó dường như hoạt động kém gần bằng mô hình hồi quy tuyến tính! Lưu ý rằng phương pháp kiểm định chéo cho phép bạn không chỉ ước tính hiệu suất của mô hình mà còn đo lường độ chính xác của ước tính này (tức là độ lệch chuẩn). Cây quyết định có RMSE khoảng 66.868, với độ lệch chuẩn khoảng 2.061. Bạn sẽ không có được thông tin này nếu chỉ sử dụng một tập dữ liệu kiểm định. Nhưng kiểm định chéo đòi hỏi phải huấn luyện mô hình nhiều lần, vì vậy nó không phải lúc nào cũng khả thi.

Nếu bạn tính toán cùng một chỉ số cho mô hình Hồi quy tuyến tính, bạn sẽ thấy rằng RMSE trung bình là 69.858 và độ lệch chuẩn là 4.182. Vì vậy, mô hình Cây quyết định dường như hoạt động tốt hơn một chút so với mô hình tuyến tính, nhưng chỉ tốt hơn một chút do hiện tượng quá khớp nghiêm trọng. Chúng ta biết có vấn đề quá khớp vì lỗi huấn luyện thấp (thực tế là bằng không) trong khi lỗi kiểm định cao.

Giờ chúng ta hãy thử mô hình cuối cùng: RandomForestRegressor. Như chúng ta sẽ thấy trong **Chương 7**, Rừng ngẫu nhiên hoạt động bằng cách huấn luyện nhiều Cây quyết định trên các tập con ngẫu nhiên của các đặc trưng, sau đó tính trung bình các dự đoán của chúng. Các mô hình được tạo thành từ nhiều mô hình khác được gọi là các mô hình kết hợp (ensemble): chúng có khả năng nâng cao hiệu suất của mô hình cơ bản (trong trường hợp này là Cây quyết định). Mã nguồn tương tự như trước:

```
from sklearn.ensemble import RandomForestRegressor

forest_reg = make_pipeline(preprocessing,

RandomForestRegressor(random_state=42))
forest_rmse = -cross_val_score(forest_reg, housing,
housing_labels,

scoring="neg_root_mean_squared_error", cv=10)
```

Hãy cùng xem những điểm số này:

```
>>> pd.Series(forest_rmse).describe()
```

số lượng 10.000000

trung 47019.561281

bình 1033.957120

độ lệch 45458.112527

chuẩn 46464.031184

tối 46967.596354

thiếu 25% 50% 75% 47325.694987

kiểu 49243.765795

dữ liệu tối đa: float64

Tuyệt vời, cái này tốt hơn nhiều: Rừng ngẫu nhiên thực sự trông rất hứa hẹn!

Nhiệm vụ này! Tuy nhiên, nếu bạn huấn luyện một RandomForest và đo RMSE

Trên tập dữ liệu huấn luyện, bạn sẽ tìm thấy khoảng 17.474: con số này thấp hơn nhiều.

Điều đó có nghĩa là vẫn còn khá nhiều hiện tượng quá khớp dữ liệu đang diễn ra. Có thể

Các giải pháp là đơn giản hóa mô hình, ràng buộc nó (tức là điều chỉnh nó), hoặc đạt được một kết quả mong muốn.

Nhiều dữ liệu huấn luyện hơn. Trước khi bạn tìm hiểu sâu hơn về Rừng ngẫu nhiên,

Tuy nhiên, bạn nên thử nhiều mẫu khác nhau từ nhiều danh mục khác nhau.

Các thuật toán Học máy (ví dụ: một số Máy hỗ trợ vectơ với)

các nhân khác nhau, và có thể là một mạng nơ-ron), mà không tốn quá nhiều chi phí.

Mất rất nhiều thời gian để tinh chỉnh các siêu tham số. Mục tiêu là chọn ra một vài (hai) trong số đó.

đến năm) mô hình đầy triển vọng.

Tinh chỉnh mô hình của bạn

Giả sử bạn hiện đã có một danh sách rút gọn các mô hình tiềm năng. Bây giờ bạn...

Cần phải tinh chỉnh chúng. Hãy cùng xem xét một vài cách bạn có thể làm điều đó.

Tìm kiếm lưới

Một lựa chọn khác là điều chỉnh các siêu tham số theo cách thủ công cho đến khi bạn...

Tìm ra sự kết hợp tối ưu giữa các giá trị siêu tham số. Điều này sẽ rất hữu ích.

Công việc tẻ nhạt, và bạn có thể không có thời gian để thử nhiều sự kết hợp khác nhau.

Thay vào đó, bạn có thể sử dụng lớp GridSearchCV của Scikit-Learn để tìm kiếm.

Bạn. Tất cả những gì bạn cần làm là cho nó biết bạn muốn sử dụng siêu tham số nào.

Thử nghiệm với các giá trị khác nhau và sử dụng phương pháp kiểm định chéo.

Để đánh giá tất cả các tổ hợp giá trị siêu tham số có thể có. Ví dụ, đoạn mã sau tìm kiếm tổ hợp giá trị siêu tham số tốt nhất cho RandomForestRegressor:

```
from sklearn.model_selection import GridSearchCV

full_pipeline =
    Pipeline([ ("preprocessing",
                preprocessing), ("random_forest", RandomForestRegressor(random_state=42)),

]) param_grid
    = [ {'preprocessing__geo__n_clusters': [5, 8,
        10], 'random_forest__max_features': [4, 6,
        8]}, {'preprocessing__geo__n_clusters': [10,
        15], 'random_forest__max_features': [6, 8, 10]},

] grid_search = GridSearchCV(full_pipeline, param_grid, cv=3,
                             scoring='neg_root_mean_squared_error')
grid_search.fit(housing, housing_labels)
```

Lưu ý rằng bạn có thể tham chiếu đến bất kỳ siêu tham số nào của bất kỳ bộ ước lượng nào trong một pipeline, ngay cả khi bộ ước lượng này nằm sâu bên trong nhiều pipeline và bộ chuyển đổi cột. Ví dụ, khi Scikit-Learn thấy "preprocessing__geo__n_clusters", nó sẽ tách chuỗi này tại dấu gạch dưới kép, sau đó tìm kiếm một bộ ước lượng có tên "preprocessing" bên trong pipeline và tìm thấy ColumnTransformer preprocessing. Sau đó, nó tìm kiếm một bộ chuyển đổi có tên "geo" bên trong ColumnTransformer này và tìm thấy bộ chuyển đổi ClusterSimilarity mà chúng ta đã sử dụng trên các thuộc tính vĩ độ và kinh độ. Tiếp theo, nó tìm thấy siêu tham số n_clusters của bộ chuyển đổi này. Tương tự, random_forest__max_features tham chiếu đến siêu tham số max_features của bộ ước lượng có tên "random_forest", tất nhiên đó là mô hình RandomForest (siêu tham số max_features sẽ được giải thích trong [Chương 7](#)).

MẸO

Việc gói gọn các bước tiền xử lý vào một pipeline của Scikit-Learn cho phép bạn điều chỉnh các siêu tham số tiền xử lý cùng với các siêu tham số của mô hình. Điều này rất tốt vì chúng thường tương tác với nhau. Ví dụ, việc tăng `n_clusters` có thể yêu cầu tăng `max_features`. Nếu việc huấn luyện các transformer trong pipeline tốn nhiều tài nguyên tính toán, bạn có thể đặt siêu tham số bộ nhớ của pipeline thành đường dẫn của thư mục bộ nhớ đệm: khi bạn huấn luyện pipeline lần đầu tiên, Scikit-Learn sẽ lưu các transformer đã huấn luyện vào thư mục này. Nếu sau đó bạn huấn luyện pipeline một lần nữa với cùng các siêu tham số, Scikit-Learn sẽ chỉ tải các transformer đã được lưu trong bộ nhớ đệm.

Có hai từ điển trong `param_grid` này, vì vậy `GridSearchCV` trước tiên sẽ đánh giá tất cả $3 \times 3 = 9$ tổ hợp giá trị siêu tham số `n_clusters` và `max_features` được chỉ định trong từ điển đầu tiên, sau đó nó sẽ thử tất cả $2 \times 3 = 6$ tổ hợp giá trị siêu tham số trong từ điển thứ hai. Vì vậy, tổng cộng quá trình tìm kiếm lưới sẽ khám phá $9 + 6 = 15$ tổ hợp giá trị siêu tham số, và nó sẽ huấn luyện pipeline 3 lần cho mỗi tổ hợp, vì chúng ta đang sử dụng phương pháp kiểm định chéo ba lần. Do đó, sẽ có tổng cộng $15 \times 3 = 45$ vòng huấn luyện! Quá trình này có thể mất một thời gian, nhưng khi hoàn tất, bạn có thể nhận được tổ hợp tham số tốt nhất như sau:

```
>>> grid_search.best_params_
{'preprocessing__geo__n_clusters': 15,
 'random_forest__max_features': 6}
```

Trong ví dụ này, mô hình tốt nhất được thu được bằng cách đặt `n_clusters` bằng 15 và `max_features` bằng 8.

MẸO

Vì 15 là giá trị tối đa được đánh giá cho `n_clusters`, có lẽ bạn nên thử tìm kiếm lại với các giá trị cao hơn; điểm số có thể tiếp tục được cải thiện.

Có thể truy cập bộ ước lượng tốt nhất bằng cách sử dụng `grid_search.best_estimator_`. Nếu `GridSearchCV` được khởi tạo.

với `refit=True` (mặc định), thì khi nó tìm thấy kết quả tốt nhất

Bộ ước lượng sử dụng phương pháp kiểm chứng chéo, nó huấn luyện lại trên toàn bộ tập dữ liệu huấn luyện. Điều này Việc cung cấp thêm dữ liệu thường là một ý tưởng hay, vì điều này có thể sẽ cải thiện khả năng của thuật toán. hiệu suất.

Bạn có thể xem điểm đánh giá bằng cách sử dụng...

`grid_search.cv_results_`. Đây là một từ điển, nhưng nếu bạn bọc nó trong

Với một `DataFrame`, bạn sẽ nhận được một danh sách đầy đủ các điểm số bài kiểm tra cho mỗi tổ hợp. của các siêu tham số và cho mỗi lần chia dữ liệu kiểm định chéo, cũng như giá trị trung bình điểm kiểm tra trên tất cả các phần.

```
>>> cv_res = pd.DataFrame(grid_search.cv_results_)
>>> cv_res.sort_values(by="mean_test_score", ascending=False,
inplace=True)
>>> [...] # thay đổi tên cột cho phù hợp với trang này và hiển thị
rmse = -score
>>> cv_res.head() # lưu ý: cột đầu tiên là ID hàng
   n_clusters max_features split0 split1 split2 mean_test_rmse
12          15           6  43460  43919  44748
44042
13          15           8  44132  44075  45010
44406
14          15          10  44374  44286  45316
44659
              10           6  44683  44655  45657
7  44999
              10           6  44683  44655  45657
9  44999
```

Điểm RMSE trung bình của mô hình tốt nhất là 44.042, tốt hơn so với trước đây.

so với điểm số bạn đạt được trước đó khi sử dụng các giá trị siêu tham số mặc định.

(con số là 47.019). Chúc mừng, bạn đã tinh chỉnh thành công!

Mẫu đẹp nhất!

Tìm kiếm ngẫu nhiên

Phương pháp tìm kiếm lưới phù hợp khi bạn chỉ cần khám phá một số lượng tương đối nhỏ.

các tổ hợp, như trong ví dụ trước, nhưng là `RandomizedSearchCV`

thường được ưu tiên hơn, đặc biệt khi không gian tìm kiếm siêu tham số là

lớn. Lớp này có thể được sử dụng tương tự như lớp GridSearchCV, nhưng thay vì thử tất cả các tổ hợp có thể, nó đánh giá một số lượng tổ hợp cố định, chọn một giá trị ngẫu nhiên cho mỗi siêu tham số ở mỗi lần lặp. Điều này nghe có vẻ đáng ngạc nhiên, nhưng cách tiếp cận này có một số lợi ích:

- Nếu một số siêu tham số của bạn là liên tục (hoặc rời rạc nhưng có nhiều giá trị khả dĩ), và bạn cho phép tìm kiếm ngẫu nhiên chạy trong, chẳng hạn, 1.000 lần lặp, thì nó sẽ khám phá 1.000 giá trị khác nhau cho mỗi siêu tham số này, trong khi tìm kiếm lưới chỉ khám phá một vài giá trị bạn đã liệt kê cho mỗi siêu tham số.
- Giả sử một siêu tham số thực sự không tạo ra nhiều khác biệt, nhưng bạn chưa biết điều đó. Nếu nó có 10 giá trị khả dĩ và bạn thêm nó vào thuật toán tìm kiếm lưới (grid search), thì quá trình huấn luyện sẽ mất thời gian gấp 10 lần. Nhưng nếu bạn thêm nó vào thuật toán tìm kiếm ngẫu nhiên (random search), nó sẽ không tạo ra bất kỳ sự khác biệt nào.
- Giả sử có 6 siêu tham số cần khám phá, mỗi siêu tham số có 10 giá trị khả dĩ, thì tìm kiếm lưới không còn lựa chọn nào khác ngoài việc huấn luyện mô hình một triệu lần, trong khi tìm kiếm ngẫu nhiên luôn có thể chạy với bất kỳ số lần lặp nào bạn chọn.

Đối với mỗi siêu tham số, bạn phải cung cấp một danh sách các giá trị có thể có, hoặc một phân bố xác suất:

```
from sklearn.model_selection import RandomizedSearchCV from
scipy.stats import randint

param_distributions = {'preprocessing__geo__n_clusters':
randint(low=3, high=50),
                      'random_forest__max_features': randint(low=2,
cao=20)}

rnd_search = RandomizedSearchCV(
    full_pipeline, param_distributions=param_distributions, n_iter=10, cv=3,

    scoring='neg_root_mean_squared_error', random_state=42)

rnd_search.fit(housing, housing_labels)
```

Scikit-Learn cũng có hai lớp tìm kiếm siêu tham số khác:

`HalvingRandomSearchCV` và `HalvingGridSearchCV`. Mục tiêu của chúng là sử dụng tài nguyên tính toán hiệu quả hơn, hoặc để huấn luyện nhanh hơn hoặc để khám phá không gian siêu tham số lớn hơn. Cách hoạt động như sau: trong vòng đầu tiên, nhiều tổ hợp siêu tham số (gọi là “ứng viên”) được tạo ra bằng cách sử dụng phương pháp lưới hoặc phương pháp ngẫu nhiên. Sau đó, các ứng viên này được sử dụng để huấn luyện các mô hình, rồi được đánh giá bằng phương pháp kiểm định chéo, như thường lệ. Tuy nhiên, quá trình huấn luyện sử dụng tài nguyên hạn chế, giúp tăng tốc đáng kể vòng đầu tiên. Theo mặc định, “tài nguyên hạn chế” có nghĩa là các mô hình được huấn luyện trên một phần nhỏ của tập dữ liệu huấn luyện. Tuy nhiên, cũng có những hạn chế khác, chẳng hạn như giảm số lần lặp huấn luyện nếu mô hình có siêu tham số để thiết lập. Sau khi tất cả các ứng viên đã được đánh giá, chỉ những ứng viên tốt nhất mới được vào vòng thứ hai, nơi chúng được cấp nhiều tài nguyên hơn để cạnh tranh. Sau một vài vòng, các ứng viên cuối cùng sẽ được đánh giá bằng toàn bộ tài nguyên. Điều này có thể giúp bạn tiết kiệm thời gian tinh chỉnh siêu tham số.

Phương pháp tập hợp

Một cách khác để tinh chỉnh hệ thống của bạn là thử kết hợp các mô hình hoạt động tốt nhất. Nhóm (hay “tập hợp”) thường sẽ hoạt động tốt hơn mô hình riêng lẻ tốt nhất—giống như Rừng ngẫu nhiên hoạt động tốt hơn các Cây quyết định riêng lẻ mà chúng dựa vào—đặc biệt nếu các mô hình riêng lẻ mắc các loại lỗi rất khác nhau. Ví dụ, bạn có thể huấn luyện và tinh chỉnh mô hình `k-Nearest Neighbors`, sau đó tạo một mô hình tập hợp chỉ dự đoán giá trị trung bình của dự đoán từ rừng ngẫu nhiên và dự đoán từ KNN. Chúng ta sẽ đề cập chi tiết hơn về chủ đề này trong [Chương 7](#).

Phân tích các mô hình tốt nhất và sai số của chúng.

Bạn thường sẽ có được những hiểu biết sâu sắc về vấn đề bằng cách xem xét các mô hình tốt nhất. Ví dụ, `RandomForestRegressor` có thể cho biết tầm quan trọng tương đối của mỗi thuộc tính để đưa ra dự đoán chính xác:

```
>>> final_model = rnd_search.best_estimator_ # bao gồm tiền xử lý >>>
feature_importances_
=
final_model["random_forest"].feature_importances_ >>>
feature_importances_.round(2) array([0.07, 0.05,
0.05, 0.01, 0.01, 0.01, 0.01, 0.01, 0.19, [...], 0.01])
```

Hãy sắp xếp các điểm số tầm quan trọng này theo thứ tự giảm dần và hiển thị chúng bên cạnh tên thuộc tính tương ứng:

```
>>> sorted(zip(feature_importances_,
...
final_model["preprocessing"].get_feature_names_out()),
...
reverse=True)
...
[(0.18694559869103852, 'log__median_income'),
(0.0748194905715524, 'cat__ocean_proximity_INLAND'),
(0.06926417748515576, 'bedrooms_ratio__bedrooms_ratio'),
(0.05446998753775219, 'rooms_per_house__rooms_per_house'),
(0.05262301809680712, 'people_per_house__people_per_house'),
(0.03819415873915732, 'geo__Cluster 0 similarity'),
[...],
(0.00015061247730531558, 'cat__ocean_proximity_NEAR VỊNH'),
(7.301686597099842e-05, 'cat__ocean_proximity_ISLAND')]
```

Với thông tin này, bạn có thể thử loại bỏ một số tính năng ít hữu ích hơn (ví dụ: dường như chỉ có một danh mục ocean_proximity thực sự hữu ích, vì vậy bạn có thể thử loại bỏ các danh mục khác).

MẸO

Bộ chuyển đổi `sklearn.feature\_selection.SelectFromModel` có thể tự động loại bỏ các đặc trưng ít hữu ích nhất cho bạn: khi bạn huấn luyện nó, nó sẽ huấn luyện một mô hình (thường là rừng ngẫu nhiên), xem xét thuộc tính `feature\_importances\_` và chọn ra các đặc trưng hữu ích nhất. Sau đó, khi bạn gọi `transform()`, nó chỉ đơn giản là loại bỏ các đặc trưng còn lại.

Bạn cũng nên xem xét các lỗi cụ thể mà hệ thống của bạn gặp phải, sau đó cố gắng hiểu tại sao nó lại gặp lỗi và cách khắc phục vấn đề là gì: thêm vào đó

Thêm các tính năng bổ sung hoặc loại bỏ những tính năng không cung cấp thông tin hữu ích, làm sạch các giá trị ngoại lệ, v.v.

Đây cũng là thời điểm tốt để đảm bảo mô hình của bạn không chỉ hoạt động tốt trung bình mà còn trên tất cả các loại quận huyện, bất kể đó là nông thôn hay thành thị, giàu hay nghèo, phía Bắc hay phía Nam, thiếu số hay không, v.v. Điều này đòi hỏi một chút công sức tạo ra các tập con của tập dữ liệu kiểm chứng cho mỗi loại, nhưng điều này rất quan trọng: nếu mô hình của bạn hoạt động kém trên toàn bộ một loại quận huyện, thì có lẽ không nên triển khai nó cho đến khi vấn đề được giải quyết, hoặc ít nhất là không nên sử dụng nó để đưa ra dự đoán cho loại đó, vì nó có thể gây hại nhiều hơn lợi.

Đánh giá hệ thống của bạn trên bộ dữ liệu thử nghiệm.

Sau một thời gian tinh chỉnh mô hình, cuối cùng bạn sẽ có một hệ thống hoạt động đủ tốt. Bạn đã sẵn sàng đánh giá mô hình cuối cùng trên tập dữ liệu thử nghiệm. Quá trình này không có gì đặc biệt; chỉ cần lấy các biến dự đoán và nhãn từ tập dữ liệu thử nghiệm, chạy mô hình cuối cùng để chuyển đổi dữ liệu và đưa ra dự đoán, sau đó đánh giá các dự đoán này:

```
X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()

final_predictions = final_model.predict(X_test)

final_rmse = mean_squared_error(y_test, final_predictions,
                                squared=False)
print(final_rmse) # in ra 41424.40026462184
```

Trong một số trường hợp, ước tính điểm về sai số tổng quát như vậy sẽ không đủ để thuyết phục bạn triển khai: điều gì sẽ xảy ra nếu nó chỉ tốt hơn 0,1% so với mô hình hiện đang được sử dụng? Bạn có thể muốn biết ước tính này chính xác đến mức nào. Để làm điều này, bạn có thể tính khoảng tin cậy 95% cho sai số tổng quát bằng cách sử dụng `scipy.stats.t.interval()`. Chúng ta nhận được một khoảng khá lớn từ 39.275 đến 43.467, và ước tính điểm trước đó của chúng ta là 41.424 nằm gần như ở giữa khoảng đó:

```
>>> from scipy import stats >>>
confidence = 0.95 >>>
squared_errors = (final_predictions - y_test) ** 2
np.sqrt(stats.t.interval(confidence, len(squared_errors) - 1, loc=squared_errors.mean(),
...                        scale=stats.sem(squared_errors)))
...
...
...
mảng([39275.40861216, 43467.27680583])
```

Nếu bạn tinh chỉnh nhiều siêu tham số, hiệu suất thường sẽ kém hơn một chút so với kết quả đo được bằng phương pháp kiểm định chéo. Điều này là do hệ thống của bạn được tinh chỉnh để hoạt động tốt trên dữ liệu kiểm định và có thể sẽ không hoạt động tốt trên các tập dữ liệu chưa biết. Tuy nhiên, trong ví dụ này thì không phải vậy vì RMSE trên tập dữ liệu kiểm tra thấp hơn RMSE trên tập dữ liệu kiểm định, nhưng khi điều đó xảy ra, bạn phải tránh việc tinh chỉnh các siêu tham số để làm cho các con số trông tốt hơn trên tập dữ liệu kiểm tra; những cải tiến đó khó có thể áp dụng chung cho dữ liệu mới.

Giờ là giai đoạn chuẩn bị trước khi triển khai dự án: bạn cần trình bày giải pháp của mình (nhấn mạnh những gì bạn đã học được, những gì hiệu quả và những gì không hiệu quả, những giả định đã được đưa ra và những hạn chế của hệ thống), ghi lại mọi thứ và tạo ra các bài thuyết trình hấp dẫn với hình ảnh trực quan rõ ràng và những câu dễ nhớ (ví dụ: “thu nhập trung bình là yếu tố dự báo giá nhà hàng đầu”). Trong ví dụ về nhà ở California này, hiệu suất cuối cùng của hệ thống không tốt hơn nhiều so với ước tính giá của các chuyên gia, vốn thường sai lệch tới 30%, nhưng việc triển khai nó vẫn có thể là một ý tưởng hay, đặc biệt nếu điều này giúp các chuyên gia có thêm thời gian để làm những công việc thú vị và hiệu quả hơn.

Khởi chạy, giám sát và bảo trì hệ thống của bạn.

Tuyệt vời, bạn đã được chấp thuận khởi chạy! Giờ bạn cần chuẩn bị giải pháp của mình cho môi trường sản xuất (ví dụ: hoàn thiện mã, viết tài liệu và kiểm thử, v.v.). Sau đó, bạn có thể triển khai mô hình của mình lên môi trường sản xuất. Cách cơ bản nhất để làm điều này là chỉ cần lưu mô hình tốt nhất mà bạn đã huấn luyện, chuyển tệp đó sang môi trường sản xuất và tải nó lên. Để lưu mô hình, bạn có thể sử dụng thư viện joblib như sau:

```
import joblib

joblib.dump(final_model, "my_california_housing_model.pkl")
```

MẸO

Thường thì việc lưu lại mọi mô hình bạn thử nghiệm là một ý tưởng hay, để bạn có thể dễ dàng quay lại bất kỳ mô hình nào mình muốn. Bạn cũng có thể lưu điểm số kiểm định chéo và có thể cả các dự đoán thực tế trên tập dữ liệu kiểm định. Điều này sẽ cho phép bạn dễ dàng so sánh điểm số giữa các loại mô hình khác nhau, và so sánh các loại lỗi mà chúng mắc phải.

Sau khi mô hình của bạn được chuyển sang môi trường sản xuất, bạn có thể tải và sử dụng nó. Để làm điều này, trước tiên bạn phải nhập bất kỳ lớp và hàm tùy chỉnh nào mà mô hình dựa vào (điều này có nghĩa là chuyển mã sang môi trường sản xuất), sau đó tải mô hình bằng joblib và sử dụng nó để đưa ra dự đoán:

```
import joblib

[...] # import KMeans, BaseEstimator, TransformerMixin, rbf_kernel, etc.

def column_ratio(X): return
    X[:, [0]] / X[:, [1]]

class ClusterSimilarity(BaseEstimator, TransformerMixin):
    [...]

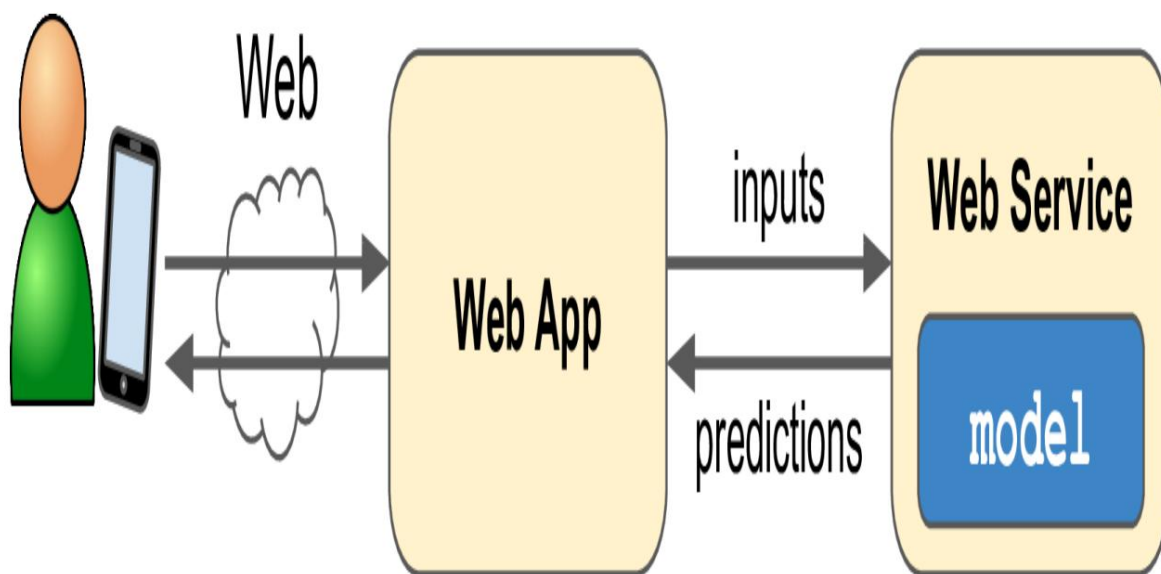
final_model_reloaded =
    joblib.load("my_california_housing_model.pkl")

new_data = [...] # một số quận mới để đưa ra dự đoán predictions =
    final_model_reloaded.predict(new_data)
```

Ví dụ, mô hình này có thể được sử dụng trong một trang web: người dùng sẽ nhập một số dữ liệu về một quận mới và nhấp vào nút “Ước tính giá”.

Thao tác này sẽ gửi một truy vấn chứa dữ liệu đến máy chủ web, máy chủ sẽ chuyển tiếp truy vấn đó đến ứng dụng web của bạn, và cuối cùng mã của bạn chỉ cần gọi phương thức predict() của mô hình (bạn nên tải mô hình khi máy chủ khởi động, chứ không phải mỗi khi mô hình được sử dụng). Ngoài ra, bạn có thể gói mô hình trong một dịch vụ web chuyên dụng mà ứng dụng web của bạn sử dụng.

Bạn có thể truy vấn thông qua API REST ¹³ (xem [Hình 2-20](#)). Điều này giúp việc nâng cấp mô hình lên các phiên bản mới dễ dàng hơn mà không làm gián đoạn ứng dụng chính. Nó cũng đơn giản hóa việc mở rộng quy mô, vì bạn có thể khởi chạy nhiều dịch vụ web khi cần và cân bằng tải các yêu cầu đến từ ứng dụng web của bạn trên các dịch vụ web này. Hơn nữa, nó cho phép ứng dụng web của bạn sử dụng bất kỳ ngôn ngữ lập trình nào, không chỉ Python.



Hình 2-20. Một mô hình được triển khai dưới dạng dịch vụ web và được sử dụng bởi một ứng dụng web.

Một chiến lược phổ biến khác là triển khai mô hình của bạn lên đám mây, ví dụ như trên Vertex AI của Google (trước đây được gọi là Google Cloud AI Platform và Google Cloud ML Engine): chỉ cần lưu mô hình của bạn bằng joblib và tải lên Google Cloud Storage (GCS), sau đó truy cập Google Cloud AI Platform và tạo một phiên bản mô hình mới, trỏ đến tệp GCS. Vậy là xong! Điều này cung cấp cho bạn một dịch vụ web đơn giản giúp cân bằng tải và mở rộng quy mô. Nó nhận các yêu cầu JSON chứa dữ liệu đầu vào (ví dụ: của một quận) và trả về các phản hồi JSON chứa các dự đoán. Sau đó, bạn có thể sử dụng dịch vụ web này trên trang web của mình (hoặc bất kỳ môi trường sản xuất nào bạn đang sử dụng). Như chúng ta sẽ thấy trong [Chương 19](#), việc triển khai các mô hình TensorFlow trên Vertex AI không khác nhiều so với việc triển khai các mô hình Scikit-Learn.

Nhưng việc triển khai không phải là điểm cuối cùng. Bạn cũng cần viết mã giám sát để kiểm tra hiệu năng hoạt động của hệ thống một cách thường xuyên.

Thiết lập khoảng thời gian và kích hoạt cảnh báo khi giá trị giảm. Giá trị có thể giảm rất nhanh, ví dụ như khi một thành phần trong cơ sở hạ tầng bị hỏng, nhưng cần lưu ý rằng nó cũng có thể giảm rất chậm, điều này dễ dàng không được chú ý trong một thời gian dài. Điều này khá phổ biến do hiện tượng "lỗi thời mô hình": nếu mô hình được huấn luyện bằng dữ liệu của năm ngoái, nó có thể không còn phù hợp với dữ liệu hiện tại.

Vì vậy, bạn cần theo dõi hiệu suất hoạt động thực tế của mô hình. Nhưng làm thế nào để làm được điều đó? Điều này phụ thuộc vào từng trường hợp. Trong một số trường hợp, hiệu suất của mô hình có thể được suy ra từ các chỉ số đầu vào. Ví dụ, nếu mô hình của bạn là một phần của hệ thống đề xuất và nó đề xuất các sản phẩm mà người dùng có thể quan tâm, thì việc theo dõi số lượng sản phẩm được đề xuất bán ra mỗi ngày rất dễ dàng. Nếu con số này giảm (so với các sản phẩm không được đề xuất), thì nguyên nhân chính có thể là do mô hình. Điều này có thể là do đường dẫn dữ liệu bị lỗi, hoặc có lẽ mô hình cần được huấn luyện lại trên dữ liệu mới (như chúng ta sẽ thảo luận ngay sau đây).

Tuy nhiên, bạn cũng có thể cần đến sự phân tích của con người để xác định hiệu suất của mô hình. Ví dụ, giả sử bạn đã huấn luyện một mô hình phân loại hình ảnh (mà chúng ta sẽ xem xét trong **Chương 3**) để phát hiện một số lỗi sản phẩm trên dây chuyền sản xuất. Làm thế nào bạn có thể nhận được cảnh báo nếu hiệu suất của mô hình giảm xuống, trước khi hàng nghìn sản phẩm lỗi được giao đến khách hàng của bạn?

Một giải pháp là gửi cho người chấm điểm một mẫu gồm tất cả các hình ảnh mà mô hình đã phân loại (đặc biệt là những hình ảnh mà mô hình không chắc chắn).

Tùy thuộc vào nhiệm vụ, người đánh giá có thể cần phải là chuyên gia, hoặc họ có thể là những người không chuyên, chẳng hạn như người lao động trên nền tảng crowdsourcing (ví dụ: Amazon Mechanical Turk). Trong một số ứng dụng, họ thậm chí có thể chính là người dùng, trả lời thông qua các cuộc khảo sát hoặc mã captcha được tái sử dụng.

14

Dù bằng cách nào, bạn cũng cần thiết lập một hệ thống giám sát (có hoặc không có người đánh giá để kiểm tra mô hình đang hoạt động), cũng như tất cả các quy trình liên quan để xác định phải làm gì trong trường hợp xảy ra lỗi và cách chuẩn bị cho chúng. Thật không may, điều này có thể tốn rất nhiều công sức. Trên thực tế, nó thường tốn nhiều công sức hơn cả việc xây dựng và huấn luyện một mô hình.

Nếu dữ liệu liên tục thay đổi, bạn sẽ cần cập nhật tập dữ liệu và huấn luyện lại mô hình thường xuyên. Có lẽ bạn nên tự động hóa toàn bộ quy trình này.

Càng nhiều càng tốt. Dưới đây là một vài việc bạn có thể tự động hóa:

- Thu thập dữ liệu mới thường xuyên và dán nhãn cho dữ liệu đó (ví dụ: sử dụng người đánh giá).
- Viết một đoạn mã để huấn luyện mô hình và tinh chỉnh các siêu tham số một cách tự động. Đoạn mã này có thể chạy tự động, ví dụ như mỗi ngày hoặc mỗi tuần, tùy thuộc vào nhu cầu của bạn.
- Hãy viết một đoạn mã khác để đánh giá cả mô hình mới và mô hình cũ trên tập dữ liệu kiểm thử đã được cập nhật, và triển khai mô hình lên môi trường sản xuất nếu hiệu suất không giảm (nếu giảm, hãy điều tra nguyên nhân). Đoạn mã này nên kiểm tra hiệu suất của mô hình trên nhiều tập con khác nhau của tập dữ liệu kiểm thử, chẳng hạn như các quận nghèo hoặc giàu, khu vực nông thôn hoặc thành thị, v.v.

Bạn cũng nên đảm bảo đánh giá chất lượng dữ liệu đầu vào của mô hình.

Đôi khi hiệu suất sẽ giảm nhẹ do tín hiệu chất lượng kém (ví dụ: cảm biến bị lỗi gửi các giá trị ngẫu nhiên, hoặc đầu ra của nhóm khác bị lỗi thời), nhưng có thể mất một thời gian trước khi hiệu suất hệ thống của bạn giảm đủ để kích hoạt cảnh báo. Nếu bạn giám sát đầu vào của mô hình, bạn có thể phát hiện điều này sớm hơn. Ví dụ, bạn có thể kích hoạt cảnh báo nếu ngày càng nhiều đầu vào thiếu một thuộc tính, hoặc nếu giá trị trung bình hoặc độ lệch chuẩn của nó lệch quá xa so với tập dữ liệu huấn luyện, hoặc một thuộc tính phân loại bắt đầu chứa các danh mục mới.

Cuối cùng, hãy đảm bảo bạn luôn sao lưu mọi mô hình mình tạo ra và có quy trình cũng như công cụ để nhanh chóng khôi phục lại mô hình trước đó, trong trường hợp mô hình mới gặp sự cố nghiêm trọng vì lý do nào đó. Việc sao lưu cũng giúp bạn dễ dàng so sánh mô hình mới với các mô hình cũ.

Tương tự, bạn nên sao lưu mọi phiên bản dữ liệu của mình để có thể khôi phục lại bộ dữ liệu trước đó nếu bộ dữ liệu mới bị lỗi (ví dụ: nếu dữ liệu mới được thêm vào chứa nhiều giá trị ngoại lai).

Việc sao lưu dữ liệu cũng cho phép bạn đánh giá bất kỳ mô hình nào dựa trên bất kỳ tập dữ liệu nào trước đó.

Như bạn thấy, Học máy đòi hỏi khá nhiều cơ sở hạ tầng.

Chương 19 thảo luận một số khía cạnh của vấn đề này, nhưng đây là một chủ đề rất rộng gọi là...